

Preprint. Under review at ACM Computing Surveys.

The Evolution of Binary Decompilation in the Modern Era: A Taxonomy, Literature Review, and Future Perspectives

OMAR ABUSABHA, Sungkyunkwan University, South Korea

SUNGJAE HWANG, Sungkyunkwan University, South Korea

Decompilation has become a foundational technique in software engineering and security analysis, and it is now advancing through the integration of modern machine learning (ML) approaches. This article presents a systematic review of 66 decompilation studies published over the past decades and develops a comprehensive taxonomy of methodologies employed in contemporary research. We further examine trends in evaluation metrics, tools, and benchmarks used to assess state-of-the-art approaches. Our review reveals key challenges, such as the lack of reliable ground truth and the absence of standardized benchmarks, which hinder rigorous comparison. Finally, we outline future research directions.

CCS Concepts: Decompiler, Binary Decompilation, Reverse Compilation

1 Introduction

Binary decompilation (a.k.a. reverse compilation)—the process of translating low-level machine code back into a high-level, source-like representation—plays a pivotal role in modern program analysis. As source code is often unavailable due to distribution practices or proprietary restrictions, decompilers offer a critical means for recovering program semantics. These reconstructed representations underpin a wide range of downstream tasks, including but not limited to malware analysis [19, 22, 50, 114, 139], binary translation [116, 117], porting [160], and code similarity analysis [29, 163]. They also aid in vulnerability detection [75, 109, 135], exploitation [75], patching [135], validation [92, 154, 156, 169] and root cause analysis [125].

Decompilation research has roots in the 1960s, beginning with Maurice Halstead’s work on the D-Neliac decompiler [73]. This work demonstrated the feasibility of transforming machine code into human-readable, source-like representations. In the 1970s, Hollander *et al.* [80] formalized decompilation into distinct stages, including initialization, scanning, parsing, construction, and code generation. Concurrently, other studies explored the usability and practical deployment of decompilers [82, 166]. By the 1990s, Cifuentes *et al.* [38] introduced a compiler-inspired pipeline organized into front-end, middle-end, and back-end phases, establishing the architectural basis of many modern traditional decompilers. Accordingly, we define the *modern era* of binary decompilation as beginning in the 1990s, when decompilation evolved toward systematic, multi-stage frameworks for recovering source-level information from binaries. This era differs from earlier research, which primarily established the feasibility, fundamental stages, and practical foundations of decompilation, by progressively advancing the architectures and techniques used to recover source-level information. A subsequent shift occurred in 2018, when Katz *et al.* [89] reimagined decompilation as a translation task, marking the emergence of modern neural decompilers.

Figure 1 provides an overview of the modern compilation and decompilation workflows. Modern compilers (*e.g.*, GCC [63] and Clang [107]) translate source code into executable binaries through multiple stages, including preprocessing (*e.g.*, semantic and syntactic validation), intermediate representation (IR) generation, optimization (*e.g.*, inlining), code generation, and linking, ultimately producing an executable binary. To reverse this process, decompilers adopt

Authors’ Contact Information: Omar Abusabha, Sungkyunkwan University, Suwon, South Korea, 404970@g.skku.edu; Sungjae Hwang, Sungkyunkwan University, Suwon, South Korea, sungjaeh@skku.edu.

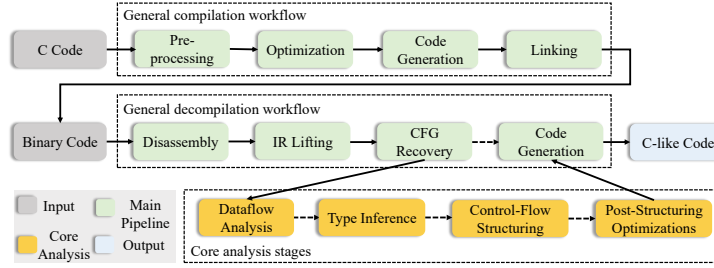


Fig. 1. Overview of modern compilation and decompilation workflows. We use directional dependencies to indicate these relationships without implying a strictly sequential pipeline.

different recovery workflows. Traditional decompilers (e.g., Hex-Rays [77] and Ghidra [118]) follow a staged architecture, starting with disassembly and IR lifting (e.g., Microcode and P-code, respectively), proceeding through control-flow graph (CFG) recovery, data-flow analysis, type inference, control-flow structuring, and post-structuring optimizations, and generating high-level source-like code. In contrast, neural decompilers (e.g., BTC [81], Neutron [100]) use learned neural models as the primary mechanism for recovering source-level representations directly from binary instructions. Moreover, while compilers optimize for execution performance and binary size, decompilers instead prioritize semantic recovery, readability, and structural clarity.

Despite decades of progress, decompilation remains a challenging process due to the lossy nature of compilation, which discards semantic cues such as variable types, function boundaries, and control structures. These challenges are further amplified by compiler diversity, platform-specific conventions, and intentional obfuscation.

Modern binary decompilation has evolved beyond a single reconstruction problem into a collection of interdependent tasks, each supported by different architectural and methodological choices. However, these developments remain scattered across studies of individual tasks, systems, and evaluation methods, making it difficult to understand how decompilation techniques have evolved, how their design choices affect the recovered code, and which directions have been explored to improve decompilation. To fill this gap, we conduct a Systematic Literature Review (SLR) [90] focused on C-language binaries. Accordingly, we formulate five primary research questions.

- **RQ1:** What architectural paradigms have shaped decompilation systems? (Section 3)
- **RQ2:** What task-oriented techniques have shaped decompilation? (Section 4)
- **RQ3:** How has the effectiveness of full-pipeline decompilers been evaluated? (Section 5)
- **RQ4:** What capabilities and metrics are used to evaluate task-oriented decompilation techniques? (Section 6)
- **RQ5:** What tools, frameworks, and benchmarks support decompilation research and evaluation? (Section 7)

The remainder of this paper is organized as follows. Section 2 describes the review methodology adopted in this study. Section 3 reviews the architectural paradigms that have shaped full-pipeline decompilation systems, while Section 4 discusses task-oriented decompilation techniques. Section 5 examines how the effectiveness of full-pipeline decompilers has been evaluated, while Section 6 reviews the capabilities and metrics used to evaluate task-oriented decompilation techniques. Section 7 surveys the tools, frameworks, and benchmarks that support decompilation research and evaluation. Section 8 outlines open challenges and future research directions, while Section 9 addresses threats to validity. Finally, Section 10 concludes the paper.

2 Research Methodology

In this section, we describe the methodology used to identify major research trends and trace the historical evolution of binary decompilation. To ensure methodological rigor, we followed the SLR guidelines by Kitchenham *et al.* [90]. The overall process is illustrated in Figure 2(a), described next.

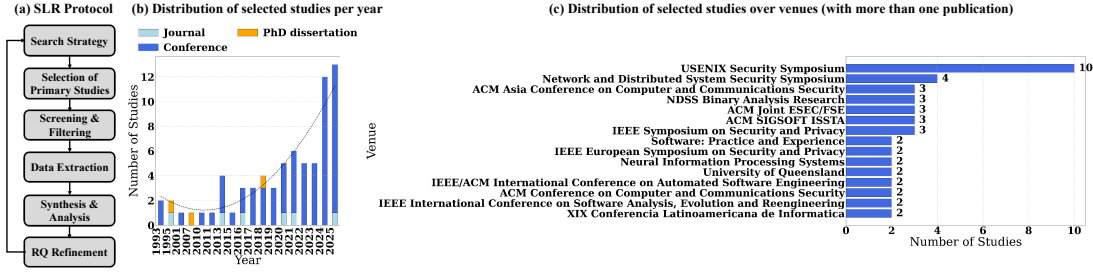


Fig. 2. Overview of binary decompilation research, including (a) the Systematic Literature Review (SLR) methodology adopted in this study following the guidelines by Kitchenham *et al.* [90], (b) the annual growth of selected studies since 1993, and (c) peer-reviewed publication venues with at least two selected studies.

2.1 Search Strategy

At this stage, we define the academic sources and search strings used to identify relevant studies on binary decompilation. We combine broad literature searches with targeted queries to improve coverage across the different decompilation paradigms and tasks.

2.1.1 Search Sources. *Google Scholar* served as the primary search engine due to its broad indexing coverage across publishers and disciplines, including gray literature, which enhanced recall. To ensure coverage of high-quality peer-reviewed sources, we supplemented the search with *IEEE Xplore* and the *ACM Digital Library*.

2.1.2 Search Strings. We initially identified relevant domain terms from foundational literature to construct comprehensive search queries. For example, foundational studies on reverse compilation [38], SSA-based decompilation [159], and human-centric binary-code decompilation [173] helped identify initial terms such as “*decompiler*”, “*type inference*”, and “*control-flow structuring*”. We then expanded and iteratively refined the keyword set, reducing the initial set of 210 candidate keywords to a final set of 64 decompilation-specific keywords.

2.2 Selection of Primary Studies

We selected the primary studies through iterative screening of titles, abstracts, and full texts, according to the inclusion and exclusion criteria defined in Table 1.

2.2.1 Screening of Retrieved Studies. We manually screened over 120 studies retrieved through the search queries to determine their relevance to binary decompilation. For instance, broad terms such as “*reverse engineering*” and “*binary analysis*” were retained in the final keyword set to improve recall. Although these terms retrieved a larger number of studies, they also returned work focused on general binary analysis and other adjacent topics.

Table 1. Inclusion and Exclusion Criteria.

Inclusion Criteria
I1: Papers published in peer-reviewed venues (<i>i.e.</i> , conferences or journals)
I2: Papers focusing on decompilation of binaries originally written in C
I3: Papers addressing either full-pipeline decompilation or task-oriented decompilation techniques covered by our taxonomy: type recovery, control-flow structuring, expression simplification, compiler idiom recovery, function recovery, variable name recovery, code summarization, and code refinement
Exclusion Criteria
E1: Non-peer-reviewed publications, including arXiv preprints, keynotes, editorials, tutorials, panel discussions, blog posts, practitioner articles, and engineering write-ups
E2: Survey, SoK, position, or taxonomy papers, unless used only for background discussion rather than inclusion in the primary study set (<i>e.g.</i> , disassembly [124], binary lifting [106], CFG recovery [162], or function identification [93])

2.2.2 Inclusion-Exclusion Criteria. To maintain the focus and relevance of our review, we applied the inclusion and exclusion criteria listed in Table 1. We included peer-reviewed studies (I1) that focus on decompiling binaries originally written in C (I2). Eligible studies address core decompilation challenges, such as type inference, control-flow structuring, and post-structuring optimization (I3). Both traditional and learning-based approaches were considered. To preserve methodological consistency, we excluded non-peer-reviewed sources, including arXiv preprints, tutorials, keynotes, blog posts, practitioner articles, and engineering write-ups. This also applies to practitioner-oriented materials for widely used decompilers, such as Ghidra [23], unless the relevant technique is documented in a peer-reviewed publication (E1). For this reason, we include Hex-Rays [68, 69] in our discussion of traditional decompiler systems where its type-inference mechanism is described in peer-reviewed work [67]. Accordingly, for actively maintained tools, our analysis reflects the designs and capabilities documented in the surveyed literature at the time of publication and does not necessarily represent their current implementations. In addition, We also excluded survey, SoK (Systematization of Knowledge), position, and taxonomy papers from the primary study set, unless they were used solely for background discussion such as disassembly [124], binary lifting [106], CFG recovery [162], or function identification [93] (E2).

2.2.3 Distribution of Studies. Figure 2(b) depicts the annual distribution of the 72 primary studies included in our review ¹. Among these studies, three are PhD dissertations, one is a SoK paper, one is a survey, and one is an empirical benchmarking study, which were used to support the review but were not included as primary technique contributions. Although decompilation efforts date back to the 1960s, the foundations of modern decompiler design emerged in the 1990s, which marks the starting point of our review focus. Research activity remained limited until around 2013, followed by a steady upward trend and a sharp increase after 2020. The highest volume of publications occurred in 2025, with 13 peer-reviewed studies published that year, reflecting heightened research activity. Most studies appeared in conference venues, which reflects the field’s emphasis on timely dissemination, particularly in areas related to security, software analysis, and program understanding.

2.2.4 Distribution of Publication Venues. Figure 2(c) shows the most common publication venues. The majority of selected studies appeared in security-oriented venues, with USENIX Security (10 studies) being the most frequent venue, followed by the Network and Distributed System Security (NDSS) Symposium. Software engineering venues such as ESEC/FSE and ISSTA also appear, demonstrating the cross-disciplinary nature of decompilation research.

¹A full list of the papers is available here: https://github.com/lion10/csur2026_binary_decompilation.git.

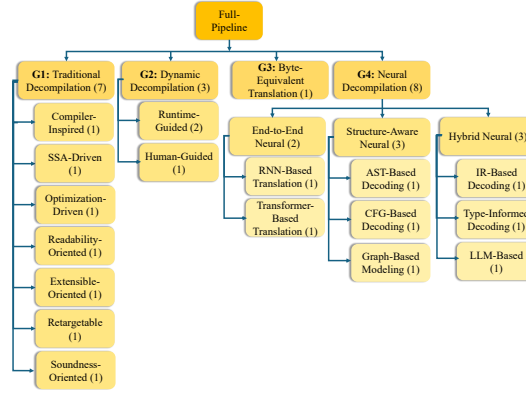


Fig. 3. Taxonomy of full-pipeline decompilation paradigms across four generations.

2.3 Data Extraction & Synthesis

After selecting the 66 primary studies, we synthesized the literature according to the five research questions. For each study, we extracted its objective, decompilation task, technical approach, evaluated capability, metrics, benchmarks, and reported limitations. We then grouped related studies into consistent categories to identify historical trends, recurring design choices, evaluation patterns, and open challenges. As a final note, we provide a high-level organization of the field and we recommend readers to refer to the original publications for an in-depth details.

3 RQ1: Research Trends in Full-Pipeline Binary Decompilation

In this section, we review full-pipeline decompilation approaches and organize them by their dominant architectural paradigms. [Figure 3](#) highlights how decompiler design has evolved across four generations (G#), discussed next.

3.1 G1: Traditional Decompilation

Traditional decompiler architectures have evolved around different design priorities over time. We organize representative systems according to their dominant architectural priority, while noting that individual systems may combine characteristics from multiple designs, as described next.

3.1.1 Compiler-Inspired. dcc [119] is an early modular decompiler that applies a compiler-inspired architecture to binary-to-C translation [38, 40]. It uses a dual-level IR comprising low-level Icode and high-level Hcode, together with idiom recognition, data-flow analysis, interval-based control-flow structuring [39], and library-signature matching. This organization established a staged decomposition in which low-level machine information is progressively transformed into higher-level program representations.

3.1.2 IR-Driven. Boomerang [16] is a retargetable, open-source decompiler that places Static Single Assignment (SSA) form at the center of its architecture [159]. Its analyses include expression propagation, register-preservation analysis, sparse data-flow-based type analysis, and modular front-end and back-end components for supporting multiple targets [158]. Unlike the staged representations of dcc, Boomerang relies on a common SSA-based representation to support multiple downstream analyses.

3.1.3 Optimization-Driven. Hex-Rays [77] is a commercial decompiler that translates architecture-specific instructions into a microcode IR and performs extensive IR simplification before code generation [67–69]. Its analyses include constant and register propagation, dead-code elimination, operand folding, high-level control-flow structuring, and FLIRT-based library recognition [78]. Its architecture therefore emphasizes progressive IR optimization before reconstructing high-level code.

3.1.4 Readability-Oriented. C-Decompiler targets readable C output from compiled programs [33, 34]. It combines a custom IR with refined stack tracing, parameter and local-variable recovery, inter-basic-block register propagation, function-signature recognition, and type and array recovery. These analyses primarily reduce redundant low-level artifacts and reconstruct higher-level source abstractions, making readability the dominant architectural objective.

3.1.5 Retargetable. RetDec [6], originating from the Lissom project [186], separates target-specific instruction decoding from reusable middle-end analyses. It uses ISAC architecture descriptions to generate instruction decoders, LLVM IR as a common intermediate representation, idiom recognition [94], control-flow structuring, and IR optimization. This separation allows the same decompilation pipeline to be adapted across architectures with limited changes to target-specific components.

3.1.6 Extensibility-Oriented. REcompile [175] is built on IDA Pro [79] as an extensible framework for static binary analysis. It combines an SSA-form IR with dead-code elimination, expression simplification, parameter detection, graph-based control-flow structuring, and basic-block transformations for reducing goto statements. Its modular design emphasizes the integration of new analysis and transformation passes as binary-analysis requirements evolve.

3.1.7 Soundness-Oriented. FoxDec [147] emphasizes sound, semantics-preserving decompilation [160]. Its architecture combines CFG recovery verified using Isabelle/HOL, symbolic execution for aggregating assembly instructions into higher-level constructs, variable mapping between symbolic memory and C variables, and type punning when precise source-level types cannot be recovered. Unlike architectures primarily optimized for readability, FoxDec prioritizes verified transformations and recompilable output.

3.2 G2: Dynamic Decompilation

Dynamic decompilation refers to approaches in which runtime evidence or analyst interaction guides the decompilation process. We group these approaches into runtime-guided and human-guided decompilation, as discussed next.

3.2.1 Runtime-Guided. One of the earliest systems in this direction was proposed by Cifuentes *et al.* [41], who integrated dynamic decompilation into the UQBT framework [122]. By executing binaries and observing runtime behavior, the system could address static challenges such as indirect jumps and code-data interleaving. Building on this concept, Botacin *et al.* [19] introduced the *Revenge* framework, combining interactive debugging (*e.g.*, via GNU Debugger) with decompilation. This allowed analysts to iteratively examine and reconstruct malware binaries, yielding recompilable code that more accurately reflects runtime semantics. In these cases, the decompiled output encodes both static structure and analyst-driven interpretation.

3.2.2 Human-Guided. Burk *et al.* [24] proposed *Decompetition*, a system for iterative, human-guided decompilation. The framework defines "perfect decompilation" as the point at which recovered source code, when compiled, yields a byte-equivalent binary. Through incremental refinement and real-time user feedback, the system allows reverse

engineers to converge on minimal and semantically accurate source representations. This process introduces an explicit stopping criterion tied to correctness, rather than subjective readability alone.

3.3 G3: Byte-Equivalent Translation Decompilation

This line of work reframes decompilation as a search problem guided by binary-level equivalence. Schulte *et al.* [140] introduced BED, which applies evolutionary algorithms to synthesize candidate programs from a corpus of human-authored source code. Each candidate is compiled and evaluated using a byte-similarity fitness function, enabling iterative refinement toward binary-equivalent outputs. While this method can recover functionally accurate and structurally simplified code, its effectiveness is limited by corpus coverage and scalability across diverse input binaries.

3.4 G4: Neural Decompilation

Neural decompilation recasts binary-to-source translation as a data-driven learning task, aiming to replace manual heuristics with model generalization. While this shift improves adaptability across architectures, it often reduces interpretability and remains limited by the quality and coverage of training data. We organize neural decompilation into three primary approaches, as described next.

3.4.1 End-to-End Neural Models. Initial efforts applied neural machine translation (NMT) to translate binary instructions directly into high-level source code using learned representations rather than architecture-specific heuristics.

- *RNN-Based Translation.* Katz *et al.* [89] applied Recurrent Neural Networks (RNNs) in a sequence-to-sequence setup to translate short assembly snippets into C. This demonstrated the feasibility of end-to-end neural decompilation, but limited structural awareness and context handling led to syntactic and semantic errors on longer sequences.
- *Transformer-Based Translation.* Beyond The C (BTC) [81] advanced end-to-end learning by employing Transformer architectures and Byte Pair Encoding (BPE) to support cross-language decompilation (e.g., C, Fortran, Go, OCaml). It treats binary and source code as plain text and improves retargetability and language independence, but may produce semantically inaccurate sketches requiring human correction.

3.4.2 Structure-Aware Neural Models. To address the limited structural context of sequence-based models, later approaches incorporated syntactic and semantic structure into the decoding process.

- *AST-Based Decoding.* Coda [61], though an end-to-end framework, introduced a two-phase decoding framework involving code sketch generation followed by iterative error correction. It combines a type-aware encoder with an AST-based decoder and compiler-guided correction to preserve syntactic structure and improve semantic consistency. However, it remains limited by complex ISAs, long programs, and static-library dependencies.
- *CFG-Based Decoding.* Neutron [100] advanced the readability and generalization of decompiled output through a novel use of basic operations as atomic translation units. Its attention-based NMT model uses finer segmentation and rule-assisted reconstruction to improve alignment between low-level and high-level semantics. However, it performs less effectively on optimized code and user-defined types.
- *Graph-Based Modeling.* N-Bref [62] introduced a structural transformer architecture tailored for non-linear code representations. It combines assembly, AST, and tree-structured representations to model control- and data-flow dependencies and separates type recovery from source-code generation. This improves structural modeling but increases dependence on accurate preprocessing such as AST and CFG recovery.

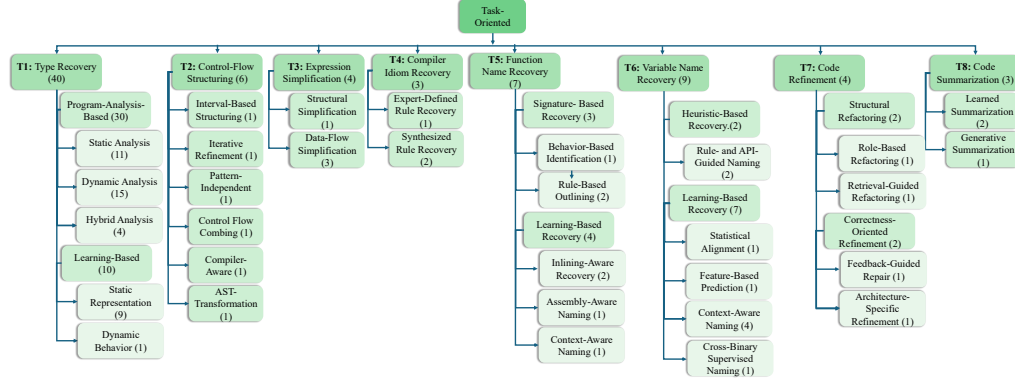


Fig. 4. Taxonomy of task-oriented decompilation techniques across eight tasks.

3.4.3 Hybrid Neural Models. Hybrid models combine neural learning with symbolic or intermediate representations to address compiler transformations that are difficult for purely neural models.

- *IR-Based Decoding.* NeurDP [30] introduced a hybrid approach that incorporated IRs into the learning process. It uses a GNN to map low-level code to a high-level IR and segments basic blocks using its Optimal Translation Unit strategy. This improves robustness to instruction reordering and dead-code elimination, but is limited to relatively simple operations.
- *Type-Informed Decoding.* SLade [5] targets semantically correct decompilation of optimized binaries across multiple Instruction Set Architectures (ISAs), including x86 and ARM. It combines a Transformer with the PsycheC symbolic type inference engine [111, 112] and uses equivalence testing to improve functional correctness. However, performance decreases on longer code and some semantically similar constructs remain difficult to distinguish.
- *LLM-Guided Refinement.* LLM4Decompile [151] leverages LLMs alongside traditional decompilers to enhance both readability and executability of decompiled code. It supports both direct binary-to-source translation through LLM4Decompile-End and refinement of Ghidra pseudo-code through LLM4Decompile-Ref. Its staged training improves compilability and executability, but dependence on reliable disassembly and decompiler output makes it vulnerable to binary obfuscation.

4 RQ2: Research Trends in Task-Oriented Binary Decompilation

Distinct from RQ1, we organize task-oriented decompilation research into eight tasks, each targeting a specific aspect of source-code recovery. Figure 4 presents this taxonomy, with each task denoted as T#. Generally speaking, Tasks 1–2 correspond to core decompiler stages, Tasks 3–6 cover post-structuring optimizations, and Tasks 7–8 operate mainly on decompiled code after code generation.

4.1 T1: Type Recovery

Type recovery encompasses type inference, prediction, and reconstruction of source-level type information that is unavailable, incomplete, or unreliable. Building on Caballero *et al.* [27], who organized 38 type-inference systems for

C/C++ binaries, 27 of which target C specifically, we extend their taxonomy with 13 additional systems. We group these approaches into two categories, described below.

4.1.1 Program-Analysis-Based Approaches. Early type-recovery research relied primarily on explicit program analysis, deriving type information from constraints, data flow, memory behavior, and runtime evidence. These methods provide interpretable recovery rules and remain the foundation of many decompilation pipelines.

- *Static Analysis.* Static approaches are primarily flow-based, deriving type information from instructions, IRs, data flow, and program constraints without executing the program. Mycroft [115], SECONDWRITE [51], RHK [138], and YM [177] recover scalar, pointer, and function-related type relations through constraint solving. Retypd [121] and PsycheC [111, 112] extend this process with unification, subtyping, and polymorphic relations. FFE [101], DIVINE [9], and TDA [153] reconstruct composite types and data structures from memory-access and control-flow relations. OSPREY [183] handles uncertain type evidence probabilistically, whereas Trex [18] uses formal reasoning to reconstruct types consistent with observed machine behavior. Static flow-based analysis provides broad program coverage and interpretable type relations, but remains sensitive to compiler optimization, aliasing, and incomplete interprocedural analysis.
- *Dynamic Analysis.* Dynamic approaches use either flow-based or value-based analysis of concrete runtime behavior. Flow-based approaches track type information through execution traces, taint propagation, and memory accesses. REWARDS [103], POINTERSCOPE [181], UNDANGLE [141], and TOP [180] infer pointer and value types by tracking runtime propagation. DYNCOMPB [70] unifies types associated with the same runtime entities, while ARTISTE [25] and HOWARD [145] reconstruct dynamic objects, recursive structures, and nested types. Value-based approaches instead inspect concrete memory values, layouts, and recurring patterns. POLYGLOT [28], AUTOFORMAT [102], TUPNI [43], DISPATCHER [32], DC [48], and DDE [144] recover data formats and aggregate layouts from observed memory behavior. MEMPICK [72] classifies memory regions from concrete values, whereas RDS [133] recovers recursive data structures through memory-shape analysis. Dynamic flow-based analysis provides precise evidence about observed value propagation, whereas value-based analysis is particularly effective for aggregate and recursive structures. Both, however, remain limited by executed paths, selected inputs, and visible runtime states.
- *Hybrid Analysis.* Hybrid approaches combine static program relations with runtime evidence through flow-based, value-based, or combined flow- and value-based analysis. In flow-based analysis, TIE [7] uses value-set analysis and lattice-based solving to recover scalar and pointer-related type relations, while WCKK [167] uses memory-access patterns to reconstruct nested data structures. In value-based analysis, DDT [88] recovers memory layouts from recurring value patterns across executions. BCR [26] combines flow-sensitive constraints, memory values, and repeated executions, representing a flow- and value-based approach that recovers both type relations and aggregate structures. Hybrid analysis reduces ambiguity by combining complementary evidence sources, but increases analysis complexity and remains dependent on representative executions.

4.1.2 Learning-Based Approaches. More recent work shifts from manually specified inference rules toward learned representations that capture recurring type patterns from large collections of binaries and decompiled code. These approaches complement program analysis by modeling semantic regularities that are difficult to encode explicitly.

- *Static Representations.* Static learning-based approaches predict types from engineered features, token sequences, or program graphs without requiring runtime execution. Debin [76] and TypeMiner [108] use engineered

features to predict variable and symbol types. EKLAVYA [36] predicts function signatures from instruction sequences, while DIRTY [35], Stir [129], and ReSym [170] predict variable types and related source-level information from ill-formed or decompiled-code sequences. TYGR [184], DRAGON [149], and GENNM [172] use graph representations to predict types from structural and data-flow relations. These approaches can recover patterns not captured by explicit constraints, but their accuracy depends on training data, representation quality, and generalization to unseen binaries.

- *Dynamic Behavior.* Dynamic learning-based approaches predict types from execution traces, runtime operands, and observed program states. StateFormer [129] models operand behavior across executions to infer runtime type information. These approaches learn semantic patterns from concrete behavior, but require representative traces and incur runtime instrumentation overhead.

4.2 T2: Control-Flow Structuring

Control-flow structuring transforms a recovered CFG into high-level constructs through semantics-preserving transformations. Behner *et al.* [11] recently systematized control-flow structuring more broadly in binary decompilation, while we exclude compiler-side methods [55, 57]. We group existing approaches into six categories, described below.

4.2.1 Interval-Based Structuring. Cifuentes *et al.* [37] introduced an interval-based approach in dcc [119]. The method recursively partitions the CFG into maximal single-entry regions and collapses recognizable loops and conditionals into structured constructs. Dominance and path analysis guide this process, while heuristics handle irregular flows such as abnormal exits. This provides a systematic foundation for CFG structuring but is less effective for irreducible control flow.

4.2.2 Iterative Refinement. To better handle irreducible regions, Schwartz *et al.* [20] introduced Phoenix on top of BAP [14]. Phoenix separates cyclic and acyclic regions and temporarily removes problematic edges through virtualization so that structuring can proceed iteratively. Removed edges are later represented using constructs such as `goto`, `break`, or `continue` to preserve semantics. This improves coverage of irregular CFGs but may still retain unstructured control when complete structuring is not possible.

4.2.3 Pattern-Independent Structuring. Yakdan *et al.* [176] introduced DREAM, built on top of IDA Pro [79], to avoid dependence on fixed structuring templates. Instead, DREAM uses topological ordering, dominance analysis, conditional refinement, and rule-based inference to restructure cyclic and acyclic regions without relying on predefined control patterns. This enables broader elimination of unstructured jumps, although the resulting structure depends on the quality of the recovered CFG and applied refinement rules.

4.2.4 Control-Flow Combing. Gussoni *et al.* [71] introduced Combing, built on top of rev.ng [137], to reduce the excessive nesting and duplication that may result from aggressive `goto` elimination. The method first decomposes the CFG into nested DAGs, restructures overlapping control paths into diamond-shaped regions, and then matches the resulting graph to C constructs such as `if`, `switch`, and `loop` statements. This produces structured control flow while controlling code complexity, but requires substantial CFG transformation before high-level reconstruction.

4.2.5 Compiler-Aware Deoptimization. Basque *et al.* [10] introduced SAILR to distinguish control-flow artifacts introduced by compiler optimizations from structures that may reflect developer intent. Built on angr [4], SAILR identifies

patterns produced by compiler transformations such as jump threading and switch lowering and applies corresponding deoptimizations to recover higher-level control flow. This compiler-aware strategy avoids indiscriminate goto elimination, but its effectiveness depends on the compiler transformations that can be recognized.

4.2.6 AST-Level Transformation. Enders *et al.* [53] proposed an AST-level approach for recovering switch statements on top of Ghidra [118]. Rather than relying on explicit jump tables or fixed CFG patterns, the method identifies clusters of comparisons over the same selector expression and rewrites equivalent if-else structures into switch statements. This allows recovery across different switch-lowering patterns, but the evaluation is limited to GCC binaries and -O0.

4.3 T3: Expression Simplification

Expression simplification rewrites low-level expressions, redundant temporaries, and opaque predicates into simpler forms to reduce syntactic complexity while preserving program semantics. We group existing approaches into two categories, described below.

4.3.1 Structural Simplification. Early work focused on removing low-level structural artifacts introduced during compilation or obfuscation. Saidi *et al.* [139] proposed transformations including stack normalization, function-body reassembly, and calling-convention rewriting. These transformations remove artifacts such as junk instructions and semantic no-ops and reconstruct more coherent local code regions. Their main advantage is the direct removal of machine-level artifacts, although they provide limited support for simplifying more complex expressions and predicates.

4.3.2 Data-Flow Simplification. Data-flow approaches use intermediate representations, often SSA-based, to propagate expressions and eliminate redundant computations. REcompile [175], for example, performs expression propagation and dead-code elimination while preserving global variables to reduce the risk of semantic loss. DREAM++ [174] further applies constant propagation and common subexpression elimination, together with symbolic reasoning through Z3 [179], to simplify complex expressions and predicates. dewolf [52] extends this direction with congruence analysis, a custom graph-based logic engine, and pointer transformation. Congruence analysis merges semantically equivalent variables or expressions, while pointer transformation rewrites low-level pointer arithmetic into higher-level constructs such as array indexing and reference-based access. These techniques can simplify expressions beyond conventional propagation and elimination, but richer symbolic and graph-based reasoning may increase computational cost and remains dependent on accurate IR and data-flow information.

4.4 T4: Compiler Idiom Recovery

Compiler idiom recovery identifies optimized instruction sequences that encode high-level operations and reconstructs their original semantics. We group existing approaches into two categories, described below.

4.4.1 Expert-Defined Pattern Recovery. Expert-defined approaches use manually crafted patterns and static analysis to recognize known compiler idioms. Lissom [94], for example, matches predefined instruction sequences in LLVM IR to recover high-level expressions. Operating on an IR can abstract architecture-specific instructions and allow rules to be reused across architectures. However, recognition remains dependent on manually maintained patterns, and verbose IR sequences can make matching difficult, particularly when compiler optimizations or instruction semantics vary substantially.

4.4.2 Synthesized Pattern Recovery. Synthesized approaches automate idiom discovery to reduce the manual effort required to construct pattern databases. PIdARCI [54] generates instruction sequences from program templates across different constant values, compiler versions, and optimization settings, then anonymizes and clusters them to identify recurring patterns. This improves coverage and facilitates updates for evolving compiler toolchains. However, transformation rules for recovering original constants still require manual derivation, and abstraction during anonymization may lose distinctions between semantically different instruction forms. dewolf [52] integrates PIdARCI within its decompilation pipeline for idiom recovery and expression simplification.

4.5 T5: Function Name Recovery

Function-name recovery reconstructs function identities or descriptive identifiers that are unavailable in stripped or optimized binaries. We group existing approaches into two categories, described below.

4.5.1 Signature- and Analysis-Based Recovery. These approaches recover known function identities or abstractions using predefined signatures, behavioral evidence, and program-analysis rules.

- *Behavior-Based Identification.* UNSTRIP [84] constructs semantic descriptors from system calls and their arguments using CFG analysis and backward slicing. Because these behaviors can remain stable despite code-layout changes, the approach is more robust than exact signature matching to some compiler transformations, but is mainly applicable to wrapper functions interacting with the operating system.
- *Rule-Based Outlining.* DREAM [176] and DREAM++ [174] identify known library-function bodies embedded within larger code regions and replace them with corresponding function calls. This restores known modular abstractions but remains limited to function bodies that can be reliably recognized.

4.5.2 Learning-Based Recovery. These approaches infer function-level semantics from learned code representations rather than relying solely on predefined signatures or analysis rules.

- *Inlining-Aware Recovery.* FUNCRE [2] uses a RoBERTa-based model trained on disassembled traces aligned with source-level markers to identify inlined library functions in optimized binaries. It supports recovery when inlining obscures conventional signatures, but its accuracy decreases when multiple functions are inlined into the same region. ERASE [182] further addresses inlining-induced modularity loss using LLM-guided analysis, recursive symbolic execution, and functional similarity to identify redundant inlined function bodies and restore function calls. This can recover higher-level modular structure, but depends on accurate control- and data-flow information from earlier decompilation stages.
- *Assembly-Aware Naming.* AsmDepictor [91] formulates function naming as translation from assembly instructions to descriptive identifiers. It uses a Transformer with byte-pair encoding to capture instruction-level semantics and reduce vocabulary sparsity. This enables naming beyond known signatures but depends on representative training data and generalization to unseen code.
- *Context-Aware Naming.* SymLM [86] extends function-name prediction by incorporating caller, callee, and instruction-level context. This provides additional semantic evidence when a function’s role cannot be inferred from its body alone, but depends on accurately recovered calling context and adequate vocabulary coverage.

4.6 T6: Variable Name Recovery

Variable-name recovery reconstructs meaningful identifiers for variables whose source-level names are unavailable after compilation. We group existing approaches into two categories, described below.

4.6.1 Heuristic-Based Recovery. Heuristic-based approaches assign variable names using explicit program patterns and known semantic cues.

- *Rule- and API-Guided Naming.* DREAM [176] and DREAM++ [174] use structural patterns and known API semantics to assign descriptive names, including loop-related variables and arguments passed to known functions. For example, variables passed to `strcpy` may be named according to their source or destination roles. These rules provide interpretable names when known patterns are available but generalize poorly beyond predefined cases.

4.6.2 Learning-Based Recovery. Learning-based approaches infer variable names from statistical, structural, and contextual representations of binary or decompiled code.

- *Statistical Alignment.* Jaffe *et al.* [85] model decompiled code as a noisy translation of source code and align decompiled variables with likely source-level identifiers. This exploits recurring naming patterns in source-code corpora but provides limited modeling of deeper program structure.
- *Feature-Based Prediction.* Debin [76] combines engineered program features with structured prediction to jointly recover variable names and types. This captures dependencies among program entities but remains dependent on manually designed features and representative training data.
- *Context-Aware Naming.* DIRE [95] incorporates lexical and AST context, while DIRTY [35] additionally uses type and data-layout information. ReSym [170] and GENNM [172] further exploit broader program context and refinement mechanisms. These richer representations improve contextual modeling but increase complexity and remain sensitive to inaccuracies in recovered program information.
- *Cross-Binary Supervised Naming.* VarBERT [123] constructs supervision by compiling the same source program into debug and stripped binaries and mapping corresponding decompiled variables. Debug information supplies source-level names for training, enabling large-scale supervision without manual annotation. The approach nevertheless depends on reliable cross-binary mapping and representative training data.

4.7 T7: Code Refinement

Code refinement transforms source-like decompiler output into cleaner, more usable, or recompilable code while aiming to preserve program semantics. Although recent approaches commonly rely on LLMs, they differ in the primary refinement objective. We group existing approaches into two categories, discussed next.

4.7.1 Structural Refactoring. These approaches improve the structure and readability of decompiled code by rewriting non-idiomatic or unnecessarily complex constructs while preserving their intended behavior.

- *Role-Based Refactoring.* DeGPT [83] uses Referee, Advisor, and Operator roles to determine whether refinement is needed, generate edits, and assess their semantic consistency. Its MSSC mechanism provides additional semantic verification, supporting comment generation and structural simplification. The approach avoids dedicated model training, but its verification can be affected by complex control flow and parsing errors.

- *Retrieval-Guided Refactoring*. PseudoFix [97] retrieves similar distorted-pseudocode/source-code pairs as in-context examples for refactoring decompiler output. It targets artifacts such as non-idiomatic gotos, poorly recovered loops, complex predicates, and redundant temporaries, with additional semantic checking through Twin-Closure. Its effectiveness, however, depends on the relevance of retrieved examples and remains limited for some complex transformations.

4.7.2 Correctness-Oriented Refinement. These approaches primarily improve the syntactic, semantic, or platform-specific correctness of decompiled code, often with the goal of producing compilable or behaviorally consistent output.

- *Feedback-Guided Repair*. DecLLM [168] uses compiler diagnostics, sanitizer reports, and runtime feedback to iteratively repair syntax, type, memory, and behavioral inconsistencies. This enables refinement toward compilable code suitable for source-level downstream analysis. However, the repair process remains constrained by errors already present in the initial decompiler output.
- *Architecture-Specific Refinement*. ARMQwen2 [104] adapts Qwen2 to ARM decompilation and introduces preprocessing for architecture-specific artifacts such as literal pools and inlined data. This specialization improves refinement of ARM-specific decompiler output, but currently focuses on individual functions and provides limited support for interprocedural context.

4.8 T8: Code Summarization

Code summarization generates natural-language descriptions of decompiled code to recover semantic information that is not explicitly represented in the binary. We group existing approaches into learned and generative summarization.

4.8.1 Learned Summarization. Learned approaches adapt models to decompiled code through task-specific fine-tuning or domain-specific pre-training. BinT5 [3] fine-tunes CodeT5 [164] on decompiled functions paired with source-level comments. This adapts source-code representations to decompiler output, but performance degrades when stripping or decompilation errors remove relevant information. HexT5 [171] instead pre-trains on pseudocode using binary-specific objectives before applying the model to summarization and related recovery tasks. This captures richer binary-specific representations, but its function-level modeling provides limited calling context.

4.8.2 Generative Summarization. Generative approaches use general-purpose models to produce natural-language descriptions of decompiled code through prompting. DeGPT [83] uses prompt-based reasoning to determine whether comment generation is needed and subsequently generates semantic comments for the decompiled code. This avoids training a dedicated summarization model, but the generated comments may still be inaccurate and depend on the model’s interpretation of the recovered code.

5 RQ3: Research Trends in Evaluation of Decompiler Effectiveness

In this section, we review how full-pipeline decompilers are evaluated in terms of their effectiveness and output quality. We organize this literature into three evaluation objectives, as shown in Figure 5. For clarity, we denote each objective as E# and discuss them next.

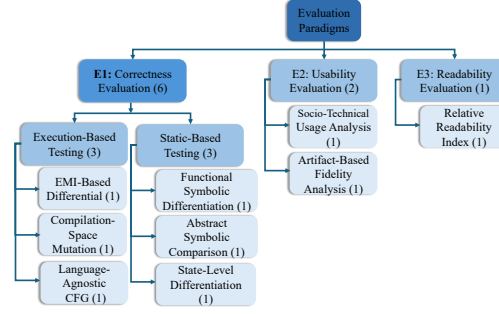


Fig. 5. Taxonomy of full-pipeline decompilation evaluation approaches across three categories.

5.1 E1: Correctness Evaluation

Correctness evaluation examines whether decompiled code preserves the functional behavior of the original binary. Existing studies evaluate correctness either through execution-based testing of runtime behavior or through static analysis of semantic equivalence. Because some approaches require compilable or analyzable output, decompiled code may first undergo syntactic repair or recompilation. We group existing approaches into two categories, described below.

5.1.1 Execution-Based Testing. Execution-based testing validates correctness by comparing the runtime behavior of decompiled code with the original binary or behaviorally equivalent program variants.

- *EMI-Based Differential Testing.* DecFuzzer [105] uses *Equivalence Modulo Inputs (EMI)* testing to generate program variants that are functionally equivalent for a given input set. By executing these variants and comparing their outputs, it detects semantic discrepancies when behavioral divergence occurs. This established execution-based differential testing for decompilers, but its evaluation is largely limited to x86 binaries compiled at -O0 and excludes several complex C constructs, including structs and arrays.
- *Compilation-Space Mutation.* Bin2Wrong [178] extends this direction by mutating both source constructs and compilation configurations. It therefore exercises multiple compilers, optimization settings, platforms, and executable formats, including ELF, PE, and Mach-O. This broader mutation space exposes configuration-specific bugs that EMI-based source variation may miss, at the cost of a larger and more complex testing space.
- *Language-Agnostic Control-Flow Testing.* FuzzFlesh [65] moves beyond source-language-specific testing by generating abstract CFGs and instantiating them as Java bytecode, .NET assembly, and x86-64 machine code. It checks whether recompiled output follows the expected runtime path through the original CFG, enabling cross-platform detection of control-flow misdecompilation. This improves portability across execution environments, but focuses primarily on control-flow correctness rather than the full semantics of decompiled programs.

5.1.2 Static-Based Testing. Static-based testing compares semantic representations of the original binary and decompiled output without relying solely on concrete executions.

- *Functional Symbolic Differentiation.* D-Helix [185] includes a RECOMPILER component that repairs and recompiles decompiler output before semantic comparison. It then uses SYMDIFF to compare symbolic models of the original lifted binary IR and the recompiled output based on functional equivalence of return semantics. D-Helix further introduces TUNER to isolate faulty decompiler heuristics by selectively toggling them during recompilation. This

enables automated root-cause analysis, but return-oriented equivalence may overlook intermediate side effects and requires successful recompilation.

- *Abstract Symbolic Comparison.* Cao *et al.* [31] avoid recompilation by abstracting optimized LLVM IR and decompiled code into comparable symbolic representations. Using *Dsmith*-generated programs with embedded checksums, the framework compares control-flow markers and path-level data semantics through concolic testing. Although they also evaluate TransRepair [98] for syntactic repair, their comparison approach does not depend on it. This enables evaluation without successful recompilation, but requires specialized test generation and symbolic traceability.
- *State-Level Differentiation.* DiscScope [143] extends comparison from end-point behavior to intermediate program states by symbolically executing the original and recompiled binaries in parallel. It localizes divergences through variables, stack offsets, literals, control-flow regions, and statements, while resynchronizing executions when their control flow diverges. This provides finer-grained diagnosis than return-level comparison, but remains dependent on recompilable decompiler output.

5.2 E2: Usability Evaluation

Usability evaluation examines how decompiler outputs and tools support reverse-engineering practice. Existing studies evaluate usability either through socio-technical analysis of analyst interactions or through human-centered examination of decompiled-code artifacts. We group existing approaches into three categories, described below.

5.2.1 Socio-Technical Usage Analysis. Votipka *et al.* [161] combine qualitative coding, statistical analysis, and social network analysis to examine community engagement with Ghidra [118] after its public release. They collected 1,590 discussion threads from StackExchange [148], Reddit [134], and Twitter [155], covering 688 users over six months. A sample of the threads was qualitatively coded along dimensions such as discussed features, conversational structure, and forms of sensemaking. Their findings show that Ghidra’s decompiler became a central focus of discussion, both as a practical reverse-engineering tool and as a target for extension through scripting. The study also shows that community support often followed a producer-consumer pattern, where a smaller group of experienced users answered questions from a larger user base. This provides evidence of real-world usability and adoption patterns, but relies on public discourse rather than directly evaluating decompiled-code artifacts.

5.2.2 Artifact-Based Fidelity Analysis. Dramko *et al.* [49] compare original and decompiled C code from Hex-Rays, Ghidra, RetDec, and angr using open coding, thematic analysis, and inter-coder agreement. This process yielded a hierarchical taxonomy comprising 15 top-level categories and 52 subcodes, covering both correctness issues (*e.g.*, incorrect return behavior and type-recovery errors) and readability concerns (*e.g.*, redundant expressions and missing type hints). The study further distinguishes between issues that are more deterministic, and may be addressed through heuristics or more accurate type information, and issues that are more context dependent, requiring external information such as macros, identifier names, or learned models.

5.3 E3: Readability Evaluation

Decompiler output may preserve program behavior while remaining difficult for analysts to understand due to structural and syntactic artifacts. Readability evaluation captures this human-facing dimension beyond functional correctness.

Table 2. Summary of evaluation capabilities reported across decompilation studies.

Capability	Representative Metrics	Representative Papers
Compactness	Compression ratio (e.g., $1 - \frac{LOC_{dec}}{LOC_{sym}}$); reduction ratio (e.g., percentage reduction in code volume); LOC reduction; number of expressions; temporary variables	DCC [40, 119], C-Decompiler [33, 34], REcompile [175], DREAM [176], ERASE [182], PseudoFix [97]
Structuredness	Goto count (e.g., number of emitted gotos); goto elimination; structured-region coverage; control-flow recovery; cyclomatic complexity (e.g., number of independent paths); nesting depth; control-flow complexity; CFG edit distance	Phoenix [20], C-Decompiler [33, 34], ERASE [182], PseudoFix [97], SAILR [10]
Readability	Cyclomatic complexity; Halstead effort; static code features; LOC, variables, casts, branches, and gotos; variable expansion rate (e.g., variables in decompiled code relative to source variables); human assessment; user preference ranking; LLM rubric score	C-Decompiler [33, 34], REcompile [175], BED [140], rev.ng COMB [71], dewolf [52], DeGPT [83], ERASE [182], LLM4Decompile [151], SAILR [10]
Correctness	Test-suite execution; re-executability; functional matching; basic-block semantic equivalence; input-output accuracy; differential execution checksum; MSSC; online judge validation; byte similarity	Phoenix [20], DREAM [176], BED [140], FoxDec [160], NeurDP [30], DeGPT [83], ERASE [182], LLM4Decompile [151], SLaDe [5]
Completeness	Recovered functions; callee/caller recovery; CFG edge coverage; feasible control-transfer coverage; type inference coverage; recovery rate	Phoenix [20], angr [142], TYGR [184]
Usability	User studies; task success rate (e.g., correctly solved analysis tasks); analyst effort; task completion time; number of errors; Likert-scale feedback; ratings; correctness of human understanding	DREAM++ [174], Decomperson [24], ERASE [182], VarBERT [123], PseudoFix [97]
Similarity	Classification Metrics (e.g., P, R, F1); Edit distance (e.g., token- or character-level edits); average edit distance; character error rate; BLEU; ROUGE-L; METEOR; token match; embedding-based semantic similarity	Katz <i>et al.</i> [89], Coda [61], BTC [81], BinT5 [3], HexT5 [171], DIRE [95], PseudoFix [97]
Recompilability	Compilation success rate (e.g., successfully compiled outputs over all outputs); multi-configuration compilation; repair success rate	FoxDec [160], LLM4Decompile [151], ARMQwen2 [104], De-cLLM [168]

5.3.1 Relative Readability Index. Eom *et al.* [56] introduced R2I, a readability metric for decompiler output that produces a normalized score between 0 and 1. R2I first applies rule-based repairs to malformed decompiler output to make it parsable. It then extracts 31 AST-based static features capturing syntactic structure and common decompilation artifacts, with feature weights partly informed by reverse-engineer surveys. The metric enables relative comparison across decompilers, versions, and obfuscation levels and correlates with human readability judgments. However, its weighting reflects surveyed user preferences and may therefore vary with analyst expertise and downstream tasks.

6 RQ4: Research Trends in Evaluating Decompilation Capabilities

In this section, we analyze how decompilation studies evaluate eight decompiler capabilities, summarized in Table 2. For each capability, we ask three questions about what it means, how it is quantified, and what it implies for decompiled code and reverse-engineering workflows. Developing an approach that simultaneously satisfies multiple capabilities is inherently challenging, as they often capture orthogonal aspects of quality, discussed next.

6.1 Compactness

Compactness refers to how much the decompiled output reduces low-level code volume and redundancy. For reverse engineers, compact code can reduce the amount of text, expressions, and temporary variables that must be inspected.

- *Code-Volume Reduction.* Dcc [40], C-Decompiler [33, 34], REcompile [175], and DREAM [176] measure compactness by comparing the size of the decompiled code with its assembly or source counterpart, often through a normalized compression ratio such as $1 - \frac{LOC_{decompiled}}{LOC_{assembly}}$. Later work extends this measurement to related indicators, including expression count and temporary-variable count [97, 182].

6.2 Structuredness

Structuredness refers to how well decompiled output recovers high-level control-flow constructs, such as conditionals and loops, instead of exposing low-level jumps. For reverse engineers, structured code reduces the need to manually trace arbitrary branches and makes program logic easier to follow.

- *Goto Count.* Phoenix [20] evaluates structuredness by counting the number of goto statements in the output. These statements often indicate that the decompiler could not recover structured control flow from the underlying CFG. Since excessive goto usage is associated with difficult-to-maintain “spaghetti code” [46], fewer goto statements generally indicate better structural recovery.
- *Control-Flow Structure.* Beyond counting unstructured jumps, structuredness can also be evaluated by comparing the recovered control-flow structure with the source-level structure. SAILR [10] uses CFG Edit Distance (CFGED) to quantify structural deviations between them. Lower edit distance indicates greater structural similarity to the source-level program. Other studies use related measures, including structured-region coverage, cyclomatic complexity, nesting depth, and control-flow similarity [10, 33, 34, 97, 182].

6.3 Readability

Readability refers to how easily decompiled code can be understood as high-level source code. For reverse engineers, readable output reduces the effort needed to infer program intent and follow control and data relationships. Because some aspects of readability may be perceived subjectively, studies combine static code metrics with human-centered assessments.

- *Structural Complexity.* C-Decompiler [33, 34] and rev.ng COMB [71] assess readability using control-flow characteristics, especially cyclomatic complexity [110], which captures the number of independent execution paths. Lower complexity generally indicates code that is easier to follow, although it does not fully capture semantic clarity.
- *Static Code Features.* BED [140] uses static code features, such as the number of characters, casts, variable declarations, and control-flow constructs, as indirect indicators of cognitive load. SAILR [10] similarly highlights lines of code, Boolean expressions, and goto statements as readability indicators.
- *Lexical Complexity.* DeGPT [83] applies Halstead’s complexity [74], which estimates cognitive effort from the number of distinct operators and operands in the recovered code. This captures how dense or verbose the output is at the token level.
- *Survey and User Assessment.* REcompile [175] and dewolf [52] evaluate readability through human judgments collected from questionnaires, interviews, or controlled user studies. These assessments capture perceived clarity, but their results may depend on participant expertise and task context.

- *Hybrid Static and Human Evaluation.* ERASE [182] combines human evaluation with static analysis. Participants inspect decompiled functions, while the system also measures properties such as lines of code, variable count, branch count, control-flow depth, and goto usage.
- *Rubric-Based LLM Evaluation.* LLM4Decomp [151] uses GPT-4o as an evaluator with a structured rubric for syntax and structural similarity between decompiled output and source code. The score ranges from 1 (poor) to 5 (excellent), providing a systematic but model-dependent readability assessment.

6.4 Correctness

Correctness refers to whether the decompiled output preserves the behavior of the original binary. For reverse engineers, correct output is essential because analysis performed on incorrect decompiled code may lead to false conclusions about program behavior.

- *Execution-Based Validation.* DREAM [176], Phoenix [20] and LLM4Decomp [151] assess correctness by inserting recovered functions back into the original source code and running high-coverage test suites. Successful execution indicates that semantic behavior is preserved.
- *Basic-Block Semantic Equivalence.* NeurDP [30] measures correctness by comparing the semantics of corresponding basic blocks in the original low-level representation and the decompiled high-level representation. Each basic block is treated as a mapping from inputs to outputs, and correctness is determined by whether both representations produce matching outputs for the same inputs.
- *Simulation-Based Semantic Checking.* DeGPT [83] applies Micro Snippet Semantic Calculation (MSSC), which simulates function logic to verify return values and side effects without requiring full program execution. This provides a lightweight correctness signal when complete recompilation or end-to-end execution is difficult.
- *Online Judge System.* ERASE [182] evaluates correctness by submitting decompiled code to the LeetCode online judge system [96], using automated I/O test case verification.

6.5 Completeness

Completeness refers to the extent to which decompilation recovers the program elements needed to represent the original binary. For reverse engineers, incomplete recovery can hide relevant functions, control-flow paths, or type information, leading to partial or misleading program understanding.

- *Function Recovery.* Phoenix [20] demonstrates that if a callee function g is not recovered, any caller function f that invokes it may also fail to decompile. This dependency illustrates how missing components can cascade and affect the recovery of related code.
- *CFG Recovery.* angr [142] captures completeness through CFG recovery, where the goal is to cover feasible control transfers in the original binary. However, broader CFG coverage may require over-approximation, which can introduce spurious edges and trade precision for completeness.
- *Type Recovery.* TYGR [184] measures type-coverage limitations by tracking how many observed variable types in real-world binaries can be represented and inferred using its fixed type vocabulary. Types outside this vocabulary are approximated using fallback casts (e.g., `struct***` $\rightarrow$ `void*`).

6.6 Usability

Usability refers to how effectively decompiled output supports reverse-engineering tasks. For reverse engineers, usable output should reduce analysis effort, improve task accuracy, and increase confidence in the recovered code.

- *Task-Based Performance.* DREAM++ [174] evaluates usability by asking participants to complete reverse-engineering tasks and measuring task success rate, completion time, and errors. ERASE [182] similarly measures analysis time before and after modularity restoration to quantify changes in manual effort. These measures capture how effectively decompiled output supports analysts in completing practical reverse-engineering tasks.
- *Subjective Feedback.* DREAM++ [174] and Decomperson [24] use participant feedback to assess perceived readability, clarity, and usefulness. Such measures capture user perception but may vary with participant expertise and task context.
- *Perceived Structural Quality.* PseudoFix [97] evaluates perceived structural quality through human judgments of consistency and reasonability. Consistency assesses whether transformed pseudocode follows structural patterns similar to a reference sample, whereas reasonability assesses whether the code appears logically coherent and well structured.

6.7 Similarity

Similarity measures how closely decompiler output matches a ground-truth source representation, identifier, summary, or code fragment. For reverse engineers, it provides a convenient proxy for output quality, but does not necessarily imply semantic correctness or readability. Different decompilation tasks therefore adopt different similarity measures.

- *Classification Metrics.* Precision, recall, and F1 score measure agreement with ground-truth labels while accounting for false positives and false negatives. They are commonly used in recovery tasks such as type inference, as in OSPREY [183].
- *Exact Match and Accuracy.* Exact match and accuracy measure whether a predicted label, identifier, or code fragment matches the reference exactly. They are used in tasks such as variable name recovery [123].
- *Edit Distance and Character Error Rate.* These metrics quantify the token- or character-level changes required to transform generated output into the reference. They are used for source-code generation and identifier recovery [81, 89, 95].
- *N-Gram-Based Similarity.* BLEU, ROUGE-L, and METEOR measure sequence overlap between generated and reference outputs. They are commonly used for code generation and summarization [3, 97, 152, 171].
- *Embedding-Based Code Similarity.* Embedding-based metrics compare semantic representations beyond surface-level token overlap. For example, PseudoFix [97] uses CodeBERT-based similarity to compare refactored pseudocode with reference code.

6.8 Recompilability

Recompilability refers to whether decompiled code can be accepted by a standard compiler without manual repair. For reverse engineers, recompilable output enables downstream uses such as testing, patching, instrumentation, and program analysis on recovered source code.

- **Compilation Success.** FoxDec [160] evaluates recompilability by compiling the recovered C code under different compiler configurations (e.g., compiler types, versions, and optimization levels). This treats recompilability as a direct pass/fail outcome, where the generated code either compiles successfully or does not. LLM4Decompile [151], ARMQwen2 [104], and DecLLM [168] similarly report compilation success after refinement processes.

7 RQ5: Research Trends in Implementation and Benchmarks

In this section, we examine the infrastructure supporting decompilation research, including implementation setups, analysis toolchains, ML architectures, benchmark construction, and compiler configurations. These elements characterize how decompilation systems are implemented and evaluated in practice.

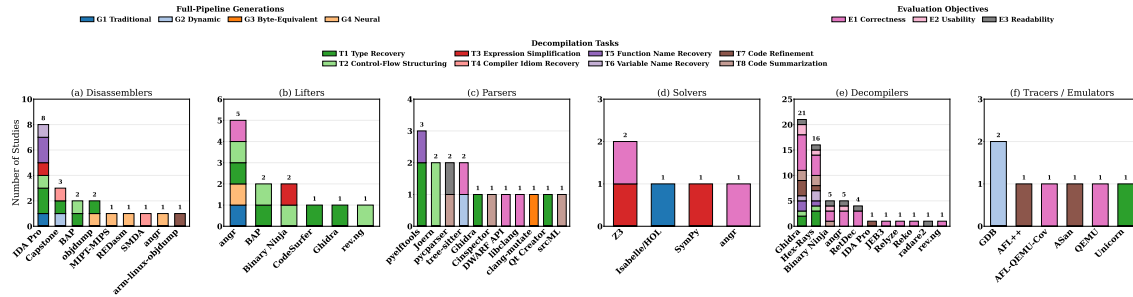


Fig. 6. Distribution of analysis tools, used across the reviewed decompilation studies. Colors indicate their use across full-pipeline generations (G#), decompilation tasks (T#), and evaluation objectives (E#).

7.1 Analysis Toolchain

Decompilation research relies on a diverse analysis toolchain to support different stages of the workflow. Figure 6 shows these tools range from static components to dynamic components. We next examine their roles, adoption across decompilation research, and key design trade-offs.

7.1.1 Disassemblers. Disassemblers convert binary code into assembly and serve as the entry point for most decompilation pipelines. Errors in instruction boundaries or control transfers can propagate to downstream analyses [124]. Figure 6(a) shows their use across G1, G2, and G4 and seven task-oriented techniques, with T1 being the most frequent. IDA Pro [79] dominates usage because of its mature interactive analysis and scripting support, while Capstone [120] provides a lightweight embeddable alternative and BAP [14] offers broader program-analysis support. These choices reflect different needs for interactive analysis, lightweight integration, and richer downstream reasoning.

7.1.2 Binary Lifters. Binary lifters translate assembly instructions into intermediate representations, enabling architecture-agnostic analyses such as data-flow tracking, type recovery, and symbolic reasoning. Their semantic accuracy is important because incomplete or incorrect instruction lifting can propagate errors to subsequent decompilation analyses [106]. Figure 6(b) shows that binary lifters are used across G1 and G4, four task-oriented techniques, and E1. They are used most frequently for T1 and T2, with four studies each. angr [4] dominates usage, appearing in five studies, due to its VEX-based IR, Python interface, and program-analysis support. BAP [14], Binary Ninja, CodeSurfer [8], rev.ng [137], and Ghidra provide alternatives with different IRs, architecture coverage, and analysis interfaces. Overall, lifter selection reflects a trade-off among semantic precision, IR expressiveness, architecture coverage, and downstream integration.

7.1.3 Parsers. Parsers extract program structure, syntax, or metadata from binaries, source code, and decompiled output for analysis, training, and evaluation. Figure 6(c) shows their use across 15 studies, with T1 and E1 being the most frequent applications. pyelftools [12] is the most common choice for binary metadata, while Joern [87], pycparser [13], and tree-sitter [21] provide graph-, AST-, and syntax-oriented representations. Parser selection therefore depends primarily on whether the analysis requires binary metadata, source syntax, or richer structural representations.

7.1.4 Solvers. Solvers support symbolic reasoning over expressions, constraints, and program states that cannot be resolved syntactically. Figure 6(d) shows their use in G1, T3, and E1. Z3 [179] dominates, while SymPy [150], angr [4], and Isabelle/HOL [127] represent algebraic simplification, symbolic execution, and formal verification, respectively. Their use trades greater semantic reasoning power for increased analysis complexity and computational cost.

7.1.5 Decompilers. Decompilers are used as experimental subjects, baselines, or sources of decompiled code for downstream analysis and refinement. Figure 6(e) shows that they form the largest infrastructure category, spanning six task-oriented techniques and all three evaluation objectives. Ghidra [118] and Hex-Rays [77] dominate usage, reflecting their mature pipelines and frequent use in comparative evaluation and downstream processing. Other decompilers broaden tool diversity for robustness and cross-decompiler comparison.

7.1.6 Tracers & Emulators. Although decompilation is primarily static, some studies use dynamic execution to obtain runtime evidence unavailable through static analysis alone. Figure 6(f) shows five studies spanning G2, T1, T7, and E1. GDB [59] is the most frequent, while emulation, instrumentation, and fuzzing tools provide complementary runtime support. Their use trades additional execution cost for direct observation of program behavior.

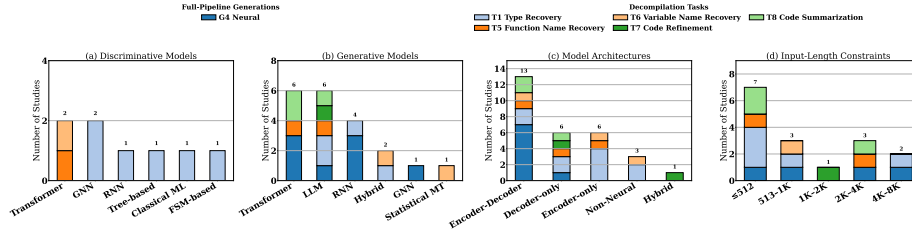


Fig. 7. Distribution of ML models by learning objectives (e.g., discriminative vs. generative), architecture, and token limits.

7.2 Model Choices

To understand how learning is incorporated into modern decompilation, we examine what models predict, which model families they adopt, how they are architecturally designed, and how much input context they can process, as summarized in Figure 7.

7.2.1 Learning Objectives. Learning-based approaches primarily follow discriminative or generative objectives, depending on whether they predict predefined outputs or generate new ones.

- **Discriminative Models.** Discriminative models classify or label elements of binary or decompiled code, making them suitable for constrained prediction tasks. As shown in Figure 7(a), eight surveyed models follow this objective. Six target T1, while T5 and T6 each account for one model. Transformers and GNNs are the most

frequent model families, with two models each, while recurrent, tree-based, finite-state, and classical machine-learning models each appear once. These choices support different forms of program context, but their predefined output spaces make them less flexible for open-ended reconstruction.

- *Generative Models.* Generative models synthesize sequences or structured representations from binary, assembly, or decompiled-code inputs. As shown in Figure 7(b), 20 surveyed models follow this objective. Eight target G4, followed by four targeting T1, three T8, two each T5 and T6, and one T7. Transformers and LLMs dominate with six models each, followed by recurrent models with four and hybrid designs with two; GNN and statistical machine translation each appear once. Generative models provide greater flexibility for open-ended reconstruction, but their larger output space can introduce syntactic or semantic errors.

7.2.2 Model Architecture. Figure 7(c) shows that encoder-decoder models dominate, with 13 studies, followed by encoder-only and decoder-only architectures with six each. Encoder-decoder designs remain common for sequence-to-sequence translation, encoder-only models support constrained prediction, and decoder-only architectures reflect the recent adoption of GPT-style LLMs for open-ended generation. Non-neural and hybrid designs are comparatively rare.

7.2.3 Input-Length Constraints. Token length determines the maximum amount of tokenized input that a model can process in a single invocation. This constraint is important in decompilation because low-level representations and long functions can expand substantially after tokenization. As shown in Figure 7(d), reported limits are concentrated in five ranges. Seven models operate at ≤ 512 tokens, three between 513 and 1K, one between 1K and 2K, three between 2K and 4K, and two between 4K and 8K. For example, BinT5 [3] and HexT5 [171] use 512-token limits, whereas LLM4Decompile [151] and DeGPT [83] use 4,096 tokens. Larger configurations include BTC [81] with 6,350 tokens and ReSym [170] with 8,192. These values represent study-specific configurations rather than fixed limits of a model family. Context capacity can vary across model versions (e.g., successive GPT-family models), while studies may also adopt smaller limits because of memory, batching, fine-tuning, or dataset constraints.

7.3 Benchmark Suites

To evaluate the effectiveness and generalizability of decompilation techniques, researchers rely on diverse benchmark datasets. These datasets are selected to reflect real-world conditions, test specific capabilities, or introduce challenging scenarios. We group these datasets into five categories, shown in Figure 8 and discussed next.

7.3.1 Open-Source Utilities. Open-source utilities provide real-world programs with accessible source code and reproducible build environments. Their availability makes them suitable for repeatable decompilation evaluation and comparison across studies. Figure 8 shows that open-source utilities are the most widely used benchmark family. Other utility programs appear in 37 studies, primarily in T1 and G4, with ten and eight studies, respectively. GNU Coreutils [60] appears in 24 studies, with its use concentrated in T1, with 11 studies, followed by T2,

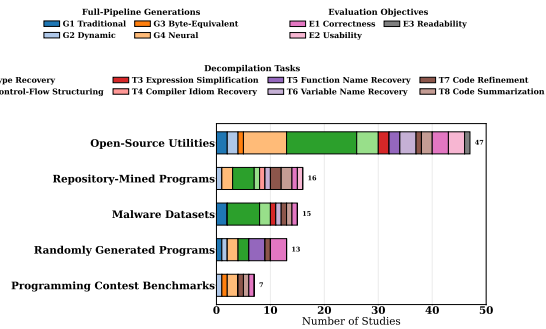


Fig. 8. Distribution of benchmark suites.

with three. Other utilities, including glibc [64], Karel [126], and Hacker’s Delight [165], provide alternative program characteristics. Overall, open-source utilities offer strong reproducibility and practical relevance, but repeated reliance on a limited set of software families may reduce benchmark diversity.

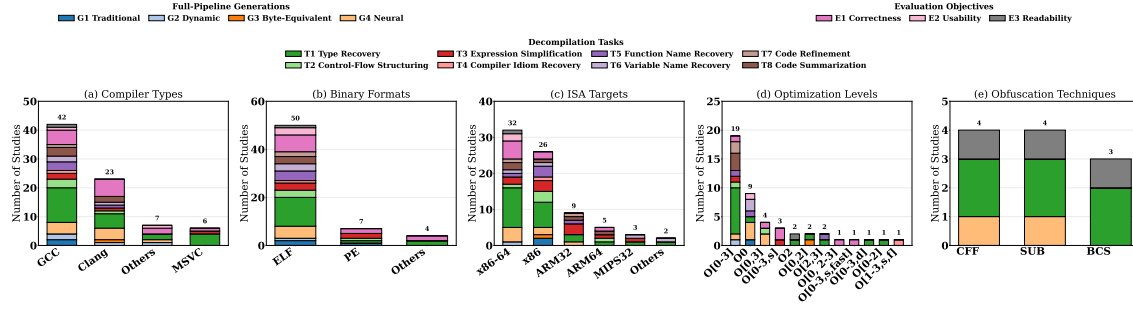
7.3.2 Large-Scale Repository Mining. Large-scale repository mining constructs benchmark corpora from public repositories or large software distributions. Its scale is particularly important for learning-based decompilation, where large numbers of binary–source pairs are required for training and evaluation. Figure 8 shows that repository-mined programs appear in 16 studies, spanning G2 and G4, six task-oriented techniques, and E1–E2. They are used most frequently for T1, with four studies, followed by G4, T7, and T8, with two studies each. These corpora are commonly obtained from repository collections such as GHTorrent [66] and GHCC [42], or software distributions such as Debian [44] and Fedora [58]. Repository mining provides scale and software diversity, but differences in collection and filtering procedures limit cross-study comparability.

7.3.3 Malware Datasets. Malware datasets expose decompilers to security-relevant and potentially irregular binary behavior. They complement conventional software benchmarks by providing evaluation settings closer to practical malware analysis. Figure 8 shows that malware datasets appear in 15 studies, spanning G1, six task-oriented techniques, and E1. Their use is concentrated in T1, with six studies, followed by G1 and T2, with two studies each. Recurring samples include SpyEye, Cridex, ZeusP2P, and Mirai, while other malware families appear more selectively. Malware datasets improve realism for security-oriented evaluation, but limited availability and inconsistent sample sharing hinder reproducibility.

7.3.4 Random Program Generators. Random program generators synthesize valid C programs for controlled and systematic decompiler testing. Their ability to generate diverse inputs makes them useful for stress testing and correctness-oriented evaluation. Figure 8 shows that randomly generated programs appear in 13 studies, spanning G1, G2, G4, three task-oriented techniques, and E1. They are used most frequently for E1 and T5, with three studies each, followed by G4 and T1, with two each. Csmith [131] is the dominant generator, while Dsmith [31] and Cfile [130] provide alternative generation strategies. Though useful for stress testing, these generators are seldom used and may poorly reflect the complexity of real-world code.

7.3.5 Programming Contest Datasets. Programming-contest datasets provide compact, standardized programs with limited external dependencies. These properties simplify dataset construction and make them particularly suitable for learning-based decompilation. Figure 8 shows that programming-contest datasets appear in seven studies, spanning G2–G4, T7–T8, and E1. G4 accounts for the largest share, with two studies, while each remaining category appears once. LeetCode [96] is the principal source, while Project Euler [132] provides primarily arithmetic-oriented programs. These datasets facilitate controlled comparison, but provide limited coverage of real-world dependencies and software complexity.

Beyond these benchmark categories, Decompile-Bench [152] represents a recent effort toward standardized benchmark construction using large-scale binary–source pairs from real-world software repositories. It therefore complements existing benchmark sources with a more systematic and reproducible construction methodology.



evidence for x86-family binaries but comparatively weaker evidence of generalization to ARM, MIPS, and other architectures.

7.4.4 Optimization Levels. Compiler optimization transforms program structure and can substantially alter the information available to a decompiler. Evaluating different optimization levels is therefore important for determining robustness to compiler transformations. Figure 9(d) shows that optimization settings are evaluated across full-pipeline approaches, task-oriented techniques, and correctness evaluation, with T1 accounting for the largest share. The standard -O0--O3 range dominates, with -O0 and -O3 being particularly common. These levels expose decompilers to substantially different amounts of compiler transformation, but inconsistent reporting and the additional use of settings such as -Os, -Og, and -Of complicate direct comparison across studies.

7.4.5 Obfuscation Techniques. Obfuscation deliberately transforms binary structure to hinder program analysis and therefore provides a challenging setting for evaluating decompilation robustness. Figure 9(e) shows that obfuscation is evaluated in only four surveyed studies. Two target T1, while G4 and E3 account for one study each, indicating substantially narrower coverage than other compilation settings. The evaluated transformations primarily include compiler-level techniques such as bogus control flow, control-flow flattening, and instruction substitution. Consequently, existing evidence primarily reflects robustness to a small set of compiler-level transformations rather than stronger real-world obfuscation such as virtualization and polymorphic transformations.

8 Open Challenges and Future Research Directions

In this section, we identified 14 critical open issues in binary decompilation, synthesized from our systematization and organized into coverage, technical, and socio-technical dimensions, as summarized in Figure 10.

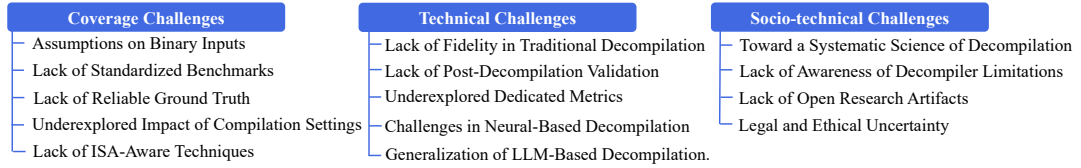


Fig. 10. Key challenges in decompilation research, organized into coverage, technical, and socio-technical dimensions.

8.1 Coverage Challenges

Coverage refers to the extent to which evaluation settings represent the diversity of binaries encountered in practice. Insufficient coverage limits the ability to generalize reported results.

8.1.1 Assumptions on Binary Inputs. Most surveyed studies evaluate well-formed binaries produced by standard compiler toolchains and generally assume that inputs are unpacked and unobfuscated. Consequently, packed executables, malformed binaries, partially recovered firmware, JIT-generated code, and binaries modified after compilation largely fall outside existing evaluation settings. Although some studies evaluate obfuscation using OLLVM [56, 128, 184], they assess robustness to transformed binaries rather than deobfuscation itself. Future studies should therefore report their input assumptions explicitly to clarify the scope and applicability of their results.

8.1.2 Lack of Standardized Benchmarks. The absence of shared, standardized and representative benchmarks hinders reproducibility and fair comparison across decompilation tools. This problem concerns not only benchmark availability, but also their representativeness and coverage of compilation effects. For example, although GNU Coreutils [60] is widely used, its programs share similar coding styles, libraries, and implementation patterns and may therefore represent only a narrow portion of real-world software. Recent efforts such as Decompile-Bench [152] improve scale and standardization through large collections of binary–source pairs, but do not by themselves ensure controlled coverage of individual compiler transformations. For example, Abusabha *et al.* [1] show that inlining behavior can be systematically tuned to expose substantially different binary structures. Future benchmarks should therefore combine broad program coverage with consistent granularity, explicit compilation metadata, and controlled variation of compiler transformations.

8.1.3 Lack of Reliable Ground Truth. Defining reliable ground truth for decompilation remains an unsolved problem. Existing strategies—such as relying on DWARF debug information [123], marker-based instrumentation [2], harnessed compilation pipelines [10], and modified compiler internals [89]—introduce substantial limitations in accuracy and realism. Debug information is often incomplete or inaccurate, especially in optimized binaries [45, 99], while synthetic or artificially constrained compilation setups fail to reflect real-world variability. Establishing standardized, high-fidelity, and diverse ground truth datasets is vital for both evaluation and training.

8.1.4 Underexplored Impact of Compilation Settings. Current decompiler evaluations rarely isolate how individual compilation settings affect the heuristics on which decompilers rely. Bin2Wrong [178] begins to address this gap by systematically varying compiler configurations, including optimization levels and individual compilation flags, to study their effects on decompiler correctness. Its findings show that evaluating only standard optimization levels can overlook failures triggered by specific compiler transformations or settings. Related compilation-space exploration techniques such as BinTuner [136], although not evaluated on decompilers, could be adapted to generate binaries that trigger decompiler heuristic failures. Future work could further use LLMs to generate targeted compilation configurations and test cases that expose when specific recovery heuristics succeed or fail.

8.1.5 Lack of ISA-Aware Techniques. ISA differences have a pronounced impact on both traditional and neural-based decompilers. For example, Fu *et al.* [61] observe that x86-64 binaries present challenges due to their verbose instruction set and implicit control-flow semantics, whereas architectures like MIPS expose branch conditions explicitly. Despite this, most decompilation approaches treat all ISAs uniformly or optimize for a single architecture. There is a pressing need for ISA-specific adaptation strategies and cross-ISA evaluation methodologies to improve the robustness and generalizability of decompilation techniques [94].

8.2 Technical Challenges

Technical challenges concern the intrinsic limitations of current decompilation techniques, including their ability to recover accurate, readable, and semantically faithful source code. Addressing these challenges is essential for improving decompiler capability and reliability.

8.2.1 Lack of Fidelity in Traditional Decompilation. Traditional decompilers primarily rely on static analysis pipelines, which often fall short in recovering high-level structure from optimized or obfuscated binaries. Type inference typically focuses on heap and stack variables, often neglecting global variables and custom data layouts [49, 184, 185]. Meanwhile, control-flow structuring algorithms [10, 176] may prioritize readability over correctness or inflate code size [71]. Consequently, outputs may appear plausible yet embed subtle semantic inaccuracies, undermining trust in decompilation

results. Addressing this fidelity gap will require integrating static, dynamic, and ML-based techniques into modular architectures capable of validating and refining their own outputs.

8.2.2 Lack of Post-Decompilation Validation. Current decompiler outputs may require additional validation to assess whether they preserve the semantics of the original binary. This issue becomes particularly important for post-decompilation techniques such as code summarization and code refinement, which further interpret or transform decompiler output. However, semantic validation after decompilation remains underexplored [19, 65, 75, 109]. Future work should investigate semantic-equivalence checking, compiler-assisted validation, and feedback mechanisms that can detect and correct discrepancies introduced during decompilation or subsequent refinement.

8.2.3 Underexplored Dedicated Metrics. In our survey, we found only one dedicated metric designed specifically for decompilation, R2I [56], which targets readability. Beyond this, evaluations typically rely on generic measures, which are not tailored to the unique challenges of decompilation. While narrow, task-specific metrics can be useful in certain contexts, they may not generalize. For example, R2I is unsuitable for malware analysis scenarios where semantic understanding outweighs code readability. Developing additional dedicated metrics would enable more precise benchmarking and foster systematic tool improvement.

8.2.4 Challenges in Neural-Based Decompilation. Our analysis identifies several recurring limitations in neural-based decompilation. As discussed previously, generative approaches enlarge the output space in which syntactic or semantic errors may occur. Moreover, we show that many surveyed models operate under limited input contexts, constraining their ability to process long functions and low-level representations. These findings motivate further work on semantic robustness and scalable modeling of larger program contexts.

8.2.5 Generalization of LLM-Based Decompilation. Our survey shows that LLMs are increasingly adopted across multiple decompilation tasks, including type recovery, function naming, code summarization, and code refinement. However, studies differ substantially in their underlying models, prompting or fine-tuning strategies, input representations, and evaluation settings. This heterogeneity makes it difficult to determine whether reported improvements generalize across models, tasks, and experimental configurations. Future work should therefore establish more systematic cross-model and cross-task evaluation of LLM-based decompilation.

8.3 Socio-technical Challenges

Socio-technical challenges concern how decompilation techniques interact with users and the research ecosystem, affecting reproducibility, adoption, collaboration, and responsible use.

8.3.1 Toward a Systematic Science of Decompilation. Beyond the primary studies reviewed in this survey, several community tools illustrate emerging support for cross-decompiler comparison and artifact synchronization. For example, mdec [17] and Decompiler Explorer [47] support comparison across decompilers, while BinSync [15] synchronizes reverse-engineering artifacts across analysis platforms. However, these capabilities provide limited insight into why a decompiler succeeds or fails, or which recovery strategy is reliable for a given binary. Future work should therefore develop adaptive and measurable decompilation workflows that expose uncertainty (e.g., warnings) and failure signals and select recovery strategies according to input characteristics.

8.3.2 Lack of Awareness of Decompiler Limitations. Decompiled outputs are used in security workflows, yet downstream analyses may not explicitly account for decompilation artifacts such as semantic inaccuracies, type errors [146],

and control-flow mistakes [11]. Future reverse-engineering pipelines should therefore incorporate error awareness, validation, and correction mechanisms when consuming decompiled code.

8.3.3 Lack of Open Research Artifacts. Roughly 48% of the studies we reviewed did not release their code, models, or datasets—highlighting a significant barrier to reproducibility in decompilation research. This lack of transparency hinders progress, limits fair comparison, and obstructs independent validation. Calling out and encouraging open practices should become a shared responsibility within the research community. Open-sourcing code, pretrained models, and standardized benchmarks is essential for fostering collaboration and accelerating innovation.

8.3.4 Legal and Ethical Uncertainty. Despite the long-standing debate around the legality of reverse engineering, we found no decompilation studies that systematically address the legal or ethical implications of their work. The legal status of decompilation varies significantly across jurisdictions, touching on issues such as copyright, fair use, and anti-circumvention laws [157]. Yet, this dimension is almost entirely absent from academic literature. Integrating those perspectives is essential to avoid unintentional misuse and to promote responsible, transparent research practices.

9 Threats to Validity

As with any systematic review, this study is subject to several limitations, including potential biases in the search, selection, and classification processes. We mitigated these risks through iterative protocol refinement, pilot reviews, and a transparent methodology, following established SLR guidelines such as those proposed by Kitchenham *et al.* [90].

- *Search Bias.* The literature search spanned from the earliest available records through December 2025, using publications indexed in Google Scholar, IEEE Xplore, and the ACM Digital Library. When publication years differed across sources, we used the year reported by Google Scholar to maintain consistent chronological ordering. Some relevant studies may have been overlooked due to limitations in search string expressiveness, keyword coverage, or indexing inconsistencies.
- *Selection Bias.* Borderline inclusion decisions required subjective judgment, which may have influenced the final dataset and affected the representativeness of our sample.
- *Classification Bias.* Study categorization was performed manually based on perceived contributions. This introduces inherent subjectivity and may have led to misclassification—particularly for studies spanning multiple research domains.
- *Generalizability.* Our findings primarily target C-language decompilation and peer-reviewed academic literature. As a result, conclusions may not generalize to other ecosystems (*e.g.*, Java or Python decompilation) or to industrial settings not represented in academic discourse.

10 Conclusion

Decompilation continues to be a critical component of modern binary analysis, yet it remains an active and evolving research area. Through this systematic review, we synthesized developments across decompilation approaches, evaluation capabilities, and technical foundations—including benchmarks and compilation settings—and identified 14 open challenges that frame the trajectory of future work. Our analysis highlights the increasing role of ML, and in particular LLMs, as promising directions for enabling more adaptive and semantically informed decompilation. We hope this survey serves as a foundation for future research, fostering the development of more robust, transparent, and practically deployable decompiler systems.

11 Ethical Concerns

This survey combined manual analysis with AI-assisted tools. NotebookLM was used to support literature organization and data extraction, while ChatGPT assisted with language refinement, manuscript organization, and consistency checking. All study selection, classification, technical analysis, and final decisions were conducted and verified by the first author.

References

- [1] Omar Abusabha, Jiyong Uhm, Tamer Abuhmed, and Hyungjoon Koo. 2026. A Deep Dive into Function Inlining and its Security Implications for ML-based Binary Analysis. In *Proceedings of the Network and Distributed System Security Symposium*.
- [2] Toufique Ahmed, Premkumar Devanbu, and Anand Ashok Sawant. 2021. Learning to Find Usages of Library Functions in Optimized Binaries. *IEEE Transactions on Software Engineering* 48, 10 (2021), 3862–3876.
- [3] Ali Al-Kaswan, Toufique Ahmed, Maliheh Izadi, Anand Ashok Sawant, Premkumar Devanbu, and Arie van Deursen. 2023. Extending source code pre-trained language models to summarise decompiled binaries. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*.
- [4] angr Project. 2015. angr binary analysis platform. <https://github.com/Angr>.
- [5] Jordi Armengol-Estapé, Jackson Woodruff, Chris Cummins, and Michael FP O’Boyle. 2024. SLDe: A Portable Small Language Model Decompiler for Optimized Assembly. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization*.
- [6] Avast Software. 2017. RetDec decompiler. <https://github.com/avast/retdec>.
- [7] Thanassis Avgerinos. 2011. TIE: Principled reverse engineering of types in binary programs. In *Proceedings of the Network and Distributed System Security Symposium*.
- [8] Gogul Balakrishnan, Radu Gruian, Thomas Reps, and Tim Teitelbaum. 2005. CodeSurfer/x86—A Platform for Analyzing x86 Executables. In *Compiler Construction (CC)*.
- [9] Gogul Balakrishnan and Thomas Reps. 2007. Divine: Discovering variables in executables. In *Proceedings of the International Workshop on Verification, Model Checking, and Abstract Interpretation*.
- [10] Zion Leonahenahe Basque, Ati Priya Bajaj, Wil Gibbs, Jude O’Kain, Derron Miao, Tiffany Bao, Adam Doupe, Yan Shoshitaishvili, and Ruoyu Wang. 2024. Ahoy Sailer! There is No Need to Dream of C: A Compiler-Aware Structuring Algorithm for Binary Decompile. In *Proceedings of the USENIX Security Symposium*.
- [11] Eva-Maria C Behner, Steffen Enders, and Elmar Padilla. 2025. Sok: No goto, no cry? the fairy tale of flawless control-flow structuring. In *Proceedings of the IEEE European Symposium on Security and Privacy*.
- [12] Eli Bendersky. 2024. pyelftools: A Pure-Python Library for Working with ELF and DWARF. <https://github.com/eliben/pyelftools>.
- [13] Eli Bendersky. 2025. pycparser: C99 parser in pure Python. <https://github.com/eliben/pycparser>.
- [14] Binary Analysis Platform (BAP). 2024. binary analysis framework. <https://github.com/BinaryAnalysisPlatform/bap>.
- [15] BinSync contributors. 2026. BinSync: A reversing plugin for cross-decompiler collaboration, built on Git. <https://github.com/binsync/binsync>.
- [16] Boomerang Decompiler Project. 2002. Boomerang decompiler. <https://github.com/BoomerangDecompiler/boomerang>.
- [17] M. Borgerson. 2022. mdec: Decompile as a Service. <https://github.com/mborgerson/mdec>.
- [18] Jay Bosamiya, Maverick Woo, and Bryan Parno. 2025. {TRex}: Practical Type Reconstruction for Binary Code. In *Proceedings of the USENIX Security Symposium*.
- [19] Marcus Botacin, Lucas Galante, Paulo de Geus, and André Grégio. 2019. Revenge Is a Dish Served Cold: Debug-Oriented Malware Decompile and Reassembly. In *Proceedings of the Reversing and Offensive-oriented Trends Symposium*.
- [20] David Brumley, JongHyup Lee, Edward J Schwartz, and Maverick Woo. 2013. Native x86 Decompile Using Semantics-Preserving Structural Analysis and Iterative Control-Flow Structuring. In *Proceedings of the USENIX Security Symposium*.
- [21] Max Brunsfeld and Tree sitter Contributors. 2025. Tree-sitter: An Incremental Parsing System for Programming Tools. <https://tree-sitter.github.io/tree-sitter/>.
- [22] Danilo Bruschi, Lorenzo Martignoni, and Mattia Monga. 2006. Using code normalization for fighting self-mutating malware. In *Proceedings of the International Symposium on Secure Software Engineering*.
- [23] Alexei Bulazel. 2019. Working With Ghidra’s P-Code To Identify Vulnerable Function Calls. <https://riverloopsecurity.com/blog/2019/05/pcode/>.
- [24] Kevin Burk, Fabio Pagani, Christopher Kruegel, and Giovanni Vigna. 2022. Decomperson: How Humans Decompile and What We Can Learn from It. In *Proceedings of the USENIX Security Symposium*.
- [25] Juan Caballero, Gustavo Grieco, Mark Marron, Zhiqiang Lin, and David Urbina. 2012. ARTISTE: automatic generation of hybrid data structure signatures from binary code executions. Technical Report TR-IMDEA-SW-2012-001. IMDEA Software Institute.
- [26] Juan Caballero, Noah M Johnson, Stephen McCamant, and Dawn Song. 2010. Binary Code Extraction and Interface Identification for Security Applications. In *Proceedings of the Network and Distributed System Security Symposium*.
- [27] Juan Caballero and Zhiqiang Lin. 2016. Type Inference on Executables. *Comput. Surveys* 48, 4 (2016), 1–35.

- [28] Juan Caballero, Heng Yin, Zhenkai Liang, and Dawn Song. 2007. Polyglot: Automatic extraction of protocol message format using dynamic binary analysis. In *Proceedings of the ACM conference on Computer and communications security*.
- [29] Aylin Caliskan, Fabian Yamaguchi, Edwin Dauber, Richard Harang, Konrad Rieck, Rachel Greenstadt, and Arvind Narayanan. 2018. When coding style survives compilation: de-anonymizing programmers from executable binaries. In *Proceedings of the Network and Distributed System Security Symposium*.
- [30] Ying Cao, Ruigang Liang, Kai Chen, and Peiwei Hu. 2022. Boosting Neural Networks to Decompile Optimized Binaries. In *Proceedings of the Annual Computer Security Applications Conference*.
- [31] Ying Cao, Runze Zhang, Ruigang Liang, and Kai Chen. 2024. Evaluating the Effectiveness of Decompilers. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*.
- [32] Martim Carbone, Weidong Cui, Long Lu, Wenke Lee, Marcus Peinado, and Xuxian Jiang. 2009. Mapping kernel objects to enable systematic integrity checking. In *Proceedings of the ACM conference on Computer and communications security*.
- [33] Gengbiao Chen, Zhengwei Qi, Shiqiu Huang, Kangqi Ni, Yudi Zheng, Walter Binder, and Haibing Guan. 2013. A Refined Decompiler to Generate C Code with High Readability. *Software: Practice and Experience* 43, 11 (2013), 1337–1358.
- [34] Gengbiao Chen, Zhuo Wang, Ruoyu Zhang, Kan Zhou, Shiqiu Huang, Kangqi Ni, Zhengwei Qi, Kai Chen, and Haibing Guan. 2010. A Refined Decompiler to Generate C Code with High Readability. In *Proceedings of the Working Conference on Reverse Engineering*.
- [35] Qibin Chen, Jeremy Lacomis, Edward J Schwartz, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. 2022. Augmenting Decompiler Output with Learned Variable Names and Types. In *Proceedings of the USENIX Security Symposium*.
- [36] Zheng Leong Chua, Shiqi Shen, Prateek Saxena, and Zhenkai Liang. 2017. Neural Nets Can Learn Function Type Signatures from Binaries. In *Proceedings of the USENIX Security Symposium*.
- [37] Cristina Cifuentes. 1993. A structuring algorithm for decompilation. In *Proceedings of the Latin American Conference on Informatics*.
- [38] Cristina Cifuentes. 1994. *Reverse compilation techniques*. Ph.D. Dissertation. Queensland University of Technology.
- [39] Cristina Cifuentes and K. John Gough. 1993. A Methodology for Decompilation. In *Proceedings of the Latin American Conference on Informatics*.
- [40] Cristina Cifuentes and K John Gough. 1995. Decompilation of Binary Programs. *Software: Practice and Experience* 25, 7 (1995), 811–829.
- [41] Cristina Cifuentes, Trent Waddington, and Mike Van Emmerik. 2001. Computer Security Analysis Through Decompilation and High-Level Debugging. In *Proceedings of the Working Conference on Reverse Engineering*.
- [42] Huze Cong. 2024. GHCC: GitHub Repository for the Codebase. <https://github.com/huzecong/ghcc>.
- [43] Weidong Cui, Marcus Peinado, Karl Chen, Helen J Wang, and Luis Irun-Briz. 2008. Tupni: Automatic reverse engineering of input formats. In *Proceedings of the ACM conference on Computer and communications security*.
- [44] Debian Project. 2025. Debian Buster package repository. <https://packages.debian.org/source/buster/>.
- [45] Giuseppe Antonio Di Luna, Davide Italiano, Luca Massarelli, Sebastian Österlund, Cristiano Giuffrida, and Leonardo Querzoni. 2021. Who's Debugging the Debuggers? Exposing Debug Information Bugs in Optimized Binaries. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [46] Edsger W Dijkstra. 1968. Letters to the Editor: Go To Statement Considered Harmful. *Commun. ACM* 11, 3 (1968), 147–148.
- [47] Dogbolt. 2024. Dogbolt: Online Decompiler Explorer. <https://dogbolt.org>.
- [48] Katerina Dolgova and Alexander Chernov. 2008. Automatic type reconstruction in disassembled c programs. In *Proceedings of the Working Conference on Reverse Engineering*.
- [49] Luke Dramko, Jeremy Lacomis, Edward J Schwartz, Bogdan Vasilescu, and Claire Le Goues. 2024. A Taxonomy of C Decompiler Fidelity Issues. In *Proceedings of the USENIX Security Symposium*.
- [50] Lukáš D'Urfina, Jakub Křoustek, and Petr Zemek. 2013. PsybOt malware: a step-by-step decompilation case study. In *Proceedings of the Working Conference on Reverse Engineering*.
- [51] Khaled ElWazeer, Kapil Anand, Aparna Kotha, Matthew Smithson, and Rajeev Barua. 2013. Scalable variable and data type detection in a binary rewriter. In *Proceedings of the ACM SIGPLAN conference on Programming language design and implementation*.
- [52] Steffen Enders, Eva-Maria C. Behner, Niklas Bergmann, Mariia Rybalka, Elmar Padilla, Er Xue Hui, Henry Low, and Nicholas Sim. 2022. dewolf: improving decompilation by leveraging user surveys. In *Proceedings of the Workshop on Binary Analysis Research*.
- [53] Steffen Enders, Eva-Maria C Behner, and Elmar Padilla. 2025. A Jump-Table-Agnostic Switch Recovery on ASTs. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution*.
- [54] Steffen Enders, Mariia Rybalka, and Elmar Padilla. 2021. PIdARCI: Using Assembly Instruction Patterns to Identify, Annotate, and Revert Compiler Idioms. In *Proceedings of International Conference on Privacy, Security and Trust*.
- [55] Felix Engel, Rainer Leupers, Gerd Ascheid, Max Ferger, and Marcel Beemster. 2011. Enhanced Structural Analysis for C Code Reconstruction from IR Code. In *Proceedings of the International Workshop on Software and Compilers for Embedded Systems*.
- [56] Haeun Eom, Dohee Kim, Sori Lim, Hyungjoon Koo, and Sungjae Hwang. 2024. R2I: a relative readability metric for decompiled code. In *Proceedings of the ACM SIGSOFT Conference on the Foundations of Software Engineering*.
- [57] Ana M Erosa and Laurie J Hendren. 1994. Taming Control Flow: A Structured Approach to Eliminating Goto Statements. In *Proceedings of the IEEE International Conference on Computer Languages*.
- [58] Fedora Project. 2025. Fedora RPM repository. <https://src.fedoraproject.org/>.
- [59] Free Software Foundation. 2021. Debugging with GDB. https://ftp.gnu.org/old-gnu/Manuals/gdb/html_mono/gdb.html.

- [60] Free Software Foundation. 2025. GNU core utilities. <https://www.gnu.org/software/coreutils/>.
- [61] Cheng Fu, Huili Chen, Haolan Liu, Xinyun Chen, Yuandong Tian, Farinaz Koushanfar, and Jishen Zhao. 2019. Coda: an end-to-end neural program decompiler. In *Proceedings of the Advances in Neural Information Processing Systems*.
- [62] Cheng Fu, Kunlin Yang, Xinyun Chen, Yuandong Tian, and Jishen Zhao. 2020. N-Bref: a high-fidelity decompiler exploiting programming structures. In *Proceedings of the International Conference on Learning Representations*.
- [63] GNU Project. 2025. GCC: GNU compiler collection. <https://gcc.gnu.org/>.
- [64] GNU Project. 2025. GNU C library (glibc). <https://www.gnu.org/software/libc/>.
- [65] Amber Gorzynski and Alastair F Donaldson. 2025. FuzzFlesh: Randomised Testing of Decompilers via Control Flow Graph-Based Program Generation. In *Proceedings of the European Conference on Object-Oriented Programming*.
- [66] Georgios Gousios. 2013. The GHTorrent Dataset and Tool Suite. In *Proceedings of the Working Conference on Mining Software Repositories*.
- [67] Ilfak Guilfanov. 2001. Simple Type System for Program Reengineering. In *Proceedings of the Working Conference on Reverse Engineering*.
- [68] Ilfak Guilfanov. 2008. Decompilers and beyond. In *Proceedings of Black Hat USA*.
- [69] Ilfak Guilfanov. 2018. Decompiler internals: microcode. In *Proceedings of Black Hat USA*.
- [70] Philip J Guo, Jeff H Perkins, Stephen McCamant, and Michael D Ernst. 2006. Dynamic inference of abstract types. In *Proceedings of the international symposium on Software testing and analysis*.
- [71] Andrea Gussoni, Alessandro Di Federico, Pietro Fezzardi, and Giovanni Agosta. 2020. A Comb for Decompiled C Code. In *Proceedings of the ACM Asia Conference on Computer and Communications Security*.
- [72] Istvan Haller, Asia Slowinska, and Herbert Bos. 2013. Mempick: High-level data structure detection in c/c++ binaries. In *Proceedings of the Working Conference on Reverse Engineering*.
- [73] Maurice H. Halstead. 1962. *Machine-Independent Computer Programming*. Spartan Books.
- [74] Maurice H. Halstead. 1977. *Elements of software science*. Elsevier Science Inc.
- [75] HyungSeok Han, JeongOh Kyea, Yonghui Jin, Jinoh Kang, Brian Pak, and Insu Yun. 2023. QueryX: Symbolic Query on Decompiled Code for Finding Bugs in COTS Binaries. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [76] Jingxuan He, Pesho Ivanov, Petar Tsankov, Veselin Raychev, and Martin Vechev. 2018. Debin: Predicting Debug Information in Stripped Binaries. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [77] Hex-Rays. 2005. Hex-Rays Decompiler. <https://hex-rays.com/decompiler/>.
- [78] Hex-Rays. 2024. FLIRT: fast library identification and recognition technology. <https://docs.hex-rays.com/user-guide/signatures/flirt>.
- [79] Hex-Rays. 2024. IDA Pro: interactive disassembler. <https://hex-rays.com/ida-pro/>.
- [80] Clifford R. Hollander. 1974. A syntax-directed approach to inverse compilation. In *Proceedings of the ACM Annual Conference*.
- [81] Iman Hosseini and Brendan Dolan-Gavitt. 2022. Beyond the C: retargetable decompilation using neural machine translation. In *Proceedings of the Workshop on Binary Analysis Research*.
- [82] Barron C. Housel and Maurice H. Halstead. 1974. A methodology for machine language decompilation. In *Proceedings of the ACM Annual Conference*.
- [83] Peiwei Hu, Ruigang Liang, and Kai Chen. 2024. DeGPT: Optimizing Decompiler Output with LLM. In *Proceedings of the Network and Distributed System Security Symposium*.
- [84] Emily R Jacobson, Nathan Rosenblum, and Barton P Miller. 2011. Labeling Library Functions in Stripped Binaries. In *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools*.
- [85] Alan Jaffe. 2017. Suggesting Meaningful Variable Names for Decompiled Code: A Machine Translation Approach. In *Proceedings of the Joint Meeting on Foundations of Software Engineering*.
- [86] Xin Jin, Kexin Pei, Jun Yeon Won, and Zhiqiang Lin. 2022. Symlm: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings. In *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security*.
- [87] Joern Development Team. 2025. Joern: code analysis platform. <https://github.com/joernio/joern>.
- [88] Changhee Jung and Nathan Clark. 2009. DDT: design and evaluation of a dynamic program analysis for optimizing data structure usage. In *Proceedings of the Annual IEEE/ACM International Symposium on Microarchitecture*.
- [89] Deborah S Katz, Jason Ruchti, and Eric Schulte. 2018. Using Recurrent Neural Networks for Decompilation. In *Proceedings of the International Conference on Software Analysis, Evolution and Reengineering*.
- [90] Staffs Keele. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report. EBSE Technical Report.
- [91] Hyunjin Kim, Jinyeong Bak, Kyunghyun Cho, and Hyungjoon Koo. 2023. A Transformer-Based Function Symbol Name Inference Model from an Assembly Language for Binary Reversing. In *Proceedings of the ACM Asia Conference on Computer and Communications Security*.
- [92] Taegyu Kim, Aolin Ding, Sriharsha Etigowni, Pengfei Sun, Jizhou Chen, Luis Garcia, Saman Zonouz, Dongyan Xu, and Dave Tian. 2022. Reverse Engineering and Retrofitting Robotic Aerial Vehicle Control Firmware Using Dispatch. In *Proceedings of the Annual International Conference on Mobile Systems, Applications and Services*.
- [93] Hyungjoon Koo, Soyeon Park, and Taesoo Kim. 2021. A Look Back on a Function Identification Problem. In *Proceedings of the Annual Computer Security Applications Conference*.
- [94] Jakub Křoustek and Fridolin Pokorný. 2013. Reconstruction of instruction idioms in a retargetable decompiler. In *Proceedings of the Federated Conference on Computer Science and Information Systems*.

- [95] Jeremy Lacomis, Pengcheng Yin, Edward Schwartz, Miltiadis Allamanis, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. 2019. Dire: A Neural Approach to Decompiled Identifier Naming. In *Proceedings of the International Conference on Automated Software Engineering*.
- [96] LeetCode. 2025. LeetCode platform. <https://leetcode.com/>.
- [97] Gangyang Li, Xiuwei Shang, Shaoyin Cheng, Junqi Zhang, Li Hu, Xu Zhu, Weiming Zhang, and Nenghai Yu. 2025. PseudoFix: Refactoring Distorted Structures in Decompiled C Pseudocode. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*.
- [98] Xueyang Li, Shangqing Liu, Ruitao Feng, Guozhu Meng, Xiaofei Xie, Kai Chen, and Yang Liu. 2022. Transrepair: Context-Aware Program Repair for Compilation Errors. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*.
- [99] Yuanbo Li, Shuo Ding, Qirun Zhang, and Davide Italiano. 2020. Debug Information Validation for Optimized Code. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*.
- [100] Ruigang Liang, Ying Cao, Peiwei Hu, and Kai Chen. 2021. Neutron: An Attention-Based Neural Decompiler. *Cybersecurity* 4, 1 (2021), 5.
- [101] Junghee Lim, Thomas Reps, and Ben Liblit. 2006. Extracting output formats from executables. In *Proceedings of the Working Conference on Reverse Engineering*.
- [102] Zhiqiang Lin, Xuxian Jiang, Dongyan Xu, and Xiangyu Zhang. 2008. Automatic protocol format reverse engineering through context-aware monitored execution. In *Proceedings of the Network and Distributed System Security Symposium*.
- [103] Zhiqiang Lin, Xiangyu Zhang, and Dongyan Xu. 2010. Automatic reverse engineering of data structures from binary execution. In *Proceedings of the Annual Information Security Symposium*.
- [104] Jiahao Liu, Jian Lin, Haoran Liu, Yonghao Wang, and Jing Jing. 2025. ARMQwen2: Enhancing C Language Decompilation on ARM Platform Using Large Language Model. In *Proceedings of the International Conference on Intelligent Computing*.
- [105] Zhibo Liu and Shuai Wang. 2020. How Far We Have Come: Testing Decompilation Correctness of C Decompilers. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*.
- [106] Zhibo Liu, Yuanyuan Yuan, Shuai Wang, and Yuyan Bao. 2022. SoK: Demystifying Binary Lifters Through the Lens of Downstream Applications. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [107] LLVM Project. 2025. Clang: C language frontend for LLVM. <https://clang.llvm.org/>.
- [108] Alwin Maier, Hugo Gascon, Christian Wressnegger, and Konrad Rieck. 2019. TypeMiner: Recovering Types in Binary Programs Using Machine Learning. In *Proceedings of the Detection of Intrusions and Malware, and Vulnerability Assessment*.
- [109] Alessandro Mantovani, Luca Compagna, Yan Shoshitaishvili, and Davide Balzarotti. 2022. The Convergence of Source Code and Binary Vulnerability Discovery—A Case Study. In *Proceedings of the ACM on Asia Conference on Computer and Communications Security*.
- [110] Thomas J. McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320.
- [111] Leandro TC Melo, Rodrigo G Ribeiro, Breno CF Guimarães, and Fernando Magno Quintão Pereira. 2020. Type inference for c: Applications to the static analysis of incomplete programs. *ACM Transactions on Programming Languages and Systems* 42, 3 (2020), 1–71.
- [112] Leandro T. C. Melo, Rodrigo G. Ribeiro, Marcus R. de Araujo, and Fernando Magno Quintão Pereira. 2017. Inference of static semantics for incomplete C programs. *Proceedings of the ACM on Programming Languages* 2, POPL (2017), 1–28.
- [113] Microsoft Corporation. 2025. Microsoft Visual C++. <https://visualstudio.microsoft.com/visual-cpp/>.
- [114] Omid Mirzaei, Roman Vasilenko, Engin Kirda, Long Lu, and Amin Kharraz. 2021. Scrutinizer: Detecting Code Reuse in Malware via Decompilation and Machine Learning. In *Proceedings of the Detection of Intrusions and Malware, and Vulnerability Assessment*.
- [115] Alan Mycroft. 1999. Type-based decompilation (or program reconstruction via type reconstruction). In *Proceedings of the European Symposium on Programming*.
- [116] Magnus O Myreen, Michael JC Gordon, and Konrad Slind. 2008. Machine-Code Verification for Multiple Architectures—An Application of Decompilation into Logic. In *Proceedings of the Formal Methods in Computer-Aided Design*.
- [117] Magnus O Myreen, Michael JC Gordon, and Konrad Slind. 2012. Decompilation into Logic—Improved. In *Proceedings of the Formal Methods in Computer-Aided Design*.
- [118] National Security Agency. 2019. Ghidra reverse engineering framework. <https://github.com/NationalSecurityAgency/ghidra>.
- [119] Nemerle. 1995. DCC decompiler. <https://github.com/nemerle/dcc>.
- [120] Nguyen Anh Quynh. 2025. Capstone: Lightweight Multi-Platform Disassembly Framework. <https://www.capstone-engine.org>.
- [121] Matt Noonan, Alexey Loginov, and David Cok. 2016. Polymorphic Type Inference for Machine Code. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*.
- [122] osFree Project. 1998. UQBT decompiler. <https://github.com/osfree-project/uqbt>.
- [123] Kuntal Kumar Pal, Ati Priya Bajaj, Pratyay Banerjee, Audrey Dutcher, Mutsumi Nakamura, Zion Leonahenahe Basque, Himanshu Gupta, Saurabh Arjun Sawant, Ujjwala Anantheswaran, Yan Shoshitaishvili, et al. 2024. Len or Index or Count, Anything but v1: Predicting Variable Names in Decompilation Output with Transfer Learning. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [124] Chengbin Pang, Ruotong Yu, Yaohui Chen, Eric Koskinen, Georgios Portokalidis, Bing Mao, and Jun Xu. 2021. SoK: All You Ever Wanted to Know About x86/x64 Binary Disassembly but Were Afraid to Ask. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [125] Younggi Park, Hwiwon Lee, Jinho Jung, Hyungjoon Koo, and Huy Kang Kim. 2024. Benzene: A practical root cause analysis system with an under-constrained state mutation. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [126] Richard E. Pattis. 1981. *Karel the robot: a gentle introduction to the art of programming*. John Wiley & Sons.
- [127] Lawrence Paulson et al. 1986. Isabelle: A Generic Proof Assistant. <https://isabelle.in.tum.de>

- [128] Kexin Pei, Jonas Guan, Matthew Broughton, Zhongtian Chen, Songchen Yao, David Williams-King, Vikas Ummadisetty, Junfeng Yang, Baishakhi Ray, and Suman Jana. 2021. StateFormer: Fine-Grained Type Recovery from Binaries Using Generative State Modeling. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.
- [129] Yaohui Peng, Jing Xie, Qiongleng Yang, Hanwen Guo, Qingan Li, Jingling Xue, and Mengting Yuan. 2023. Statistical Type Inference for Incomplete Programs. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.
- [130] Cfile Project. 2024. Cfile: C Source File Abstraction. <https://github.com/cogu/cfile>.
- [131] Csmith Project. 2024. Csmith: A Random Generator of C Programs. <https://github.com/csmith-project/csmith>.
- [132] Project Euler. 2025. Project Euler. <https://projecteuler.net/>.
- [133] Easwaran Raman and David I August. 2005. Recursive data structure profiling. In *Proceedings of the workshop on Memory system performance*.
- [134] Reddit. n.d.. Reddit. <https://www.reddit.com>.
- [135] Pemma Reiter, Hui Jun Tay, Westley Weimer, Adam Doupé, Ruoyu Wang, and Stephanie Forrest. 2024. Automatically Mitigating Vulnerabilities in Binary Programs via Partially Recompilable Decompilation. *IEEE Transactions on Dependable and Secure Computing* 22, 3 (2024), 2270–2282.
- [136] Xiaolei Ren, Michael Ho, Jiang Ming, Yu Lei, and Li Li. 2021. Unleashing the hidden power of compiler optimization on binary code difference: An empirical study. In *Proceedings of the ACM SIGPLAN International Conference on Programming Language Design and Implementation*.
- [137] rev.ng. 2016. rev.ng binary analysis framework. <https://rev.ng/>.
- [138] Ed Robbins, Jacob M Howe, and Andy King. 2013. Theory propagation and rational-trees. In *Proceedings of the Symposium on Principles and Practice of Declarative Programming*.
- [139] Hassen Saidi, Phillip Porras, and Vinod Yegneswaran. 2010. Experiences in malware binary deobfuscation. In *Proceedings of the Virus Bulletin Conference*.
- [140] Eric Schulte, Jason Ruchti, Matt Noonan, David Ciarletta, and Alexey Loginov. 2018. Evolving Exact Decompilation. In *Proceedings of the Workshop on Binary Analysis Research*.
- [141] Michael Schwarz, Daniel Gruss, Moritz Lipp, Clémentine Maurice, Thomas Schuster, Anders Fogh, and Stefan Mangard. 2018. Automated detection, exploitation, and elimination of double-fetch bugs using modern cpu features. In *Proceedings of the Asia Conference on Computer and Communications Security*.
- [142] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, et al. 2016. SoK: (State of) the Art of War: Offensive Techniques in Binary Analysis. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [143] Melih Sirlanci, Carter Yagemann, and Zhiqiang Lin. 2025. An Empirical Study of C Decompilers: Performance Metrics and Error Taxonomy. In *Proceedings of the ACM Asia Conference on Computer and Communications Security*.
- [144] Asia Slowinska, Traian Stancescu, and Herbert Bos. 2010. DDE: dynamic data structure excavation. In *Proceedings of the ACM asia-pacific workshop on Workshop on systems*.
- [145] Asia Slowinska, Traian Stancescu, and Herbert Bos. 2011. Howard: A Dynamic Excavator for Reverse Engineering Data Structures.. In *Proceedings of the Network and Distributed System Security Symposium*.
- [146] Vedant Soni, Audrey Dutcher, Tiffany Bao, and Ruoyu Wang. 2025. Benchmarking Binary Type Inference Techniques in Decompilers. In *Proceedings of the Workshop on Software Understanding and Reverse Engineering*.
- [147] SSRG-VT. 2020. FoxDec decompiler. <https://github.com/ssrg-vt/FoxDec>.
- [148] Stack Exchange. n.d.. Stack Exchange. <https://stackexchange.com>.
- [149] Caleb Stewart, Rhonda K Gaede, and Jeffrey H Kulick. 2025. DRAGON: Predicting Decompiled Variable Data Types with Learned Confidence Estimates. In *Proceedings of the Workshop on Binary Analysis Research*.
- [150] SymPy Project. 2025. SymPy: symbolic mathematics library for Python. <https://www.sympy.org/en/index.html>.
- [151] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. LLM4Decompile: Decompiling Binary Code with Large Language Models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*.
- [152] Hanzhuo Tan, Xiaolong Tian, Hanrui Qi, Jiaming Liu, Zuchen Gao, Siyi Wang, Qi Luo, Jing Li, and Yuqun Zhang. 2025. Decompile-bench: Million-scale binary-source function pairs for real-world binary decompilation. In *Proceedings of the Neural Information Processing Systems*.
- [153] Katerina Troshina, Yegor Derevenets, and Alexander Chernov. 2010. Reconstruction of composite types for decompilation. In *Proceedings of the IEEE Working conference on source code analysis and manipulation*.
- [154] Ryan Tsang, Doreen Joseph, Soheil Salehi, Prasant Mohapatra, Houman Homayoun, et al. 2024. FFxE: Dynamic Control Flow Graph Recovery for Embedded Firmware Binaries. In *Proceedings of the USENIX Security Symposium*.
- [155] Twitter. n.d.. Twitter. <https://twitter.com>.
- [156] Dimitrios Tychalas, Hadjer Benkraouda, and Michail Maniatakis. 2021. ICSFuzz: Manipulating I/Os and Repurposing Binary Code to Enable Instrumented Fuzzing in ICS Control Applications. In *Proceedings of the USENIX Security Symposium*.
- [157] University of Cambridge Computer Laboratory. 2000. The Ethics of Decompilation. <https://www.cl.cam.ac.uk/teaching/2000/OptComp/local/ethics.html>.
- [158] Michael van Emmerik and Trent Waddington. 2004. Using a decompiler for real-world source recovery. In *Proceedings of the Working Conference on Reverse Engineering*.
- [159] Michael James van Emmerik. 2007. *Static single assignment for decompilation*. Ph.D. Dissertation. University of Queensland.

- [160] Freek Verbeek, Pierre Olivier, and Binoy Ravindran. 2020. Sound C Code Decompilation for a Subset of x86-64 Binaries. In *Proceedings of the Software Engineering and Formal Methods*.
- [161] Daniel Votipka, Mary Nicole Punzalan, Seth M Rabin, Yla Tausczik, and Michelle L Mazurek. 2021. An Investigation of Online Reverse Engineering Community Discussions in the Context of Ghidra. In *Proceedings of the IEEE European Symposium on Security and Privacy*.
- [162] Qianjin Wang, Xiangdong Li, Chong Yue, and Yuchen He. 2023. A Survey of Control Flow Graph Recovery for Binary Code. In *Proceedings of the CCF National Conference of Computer Applications*.
- [163] Xiaowei Wang, Zimu Yuan, Yang Xiao, Liyan Wang, Yican Yao, Haiming Chen, and Wei Huo. 2023. Decompilation Based Deep Binary-Source Function Matching. In *Proceedings of the International Conference on Science of Cyber Security*.
- [164] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.
- [165] Jr. Warren, Henry S. 2002. *Hacker's delight*. Addison-Wesley.
- [166] Maria F. Weller. 1974. A pragmatic look at decompilers. In *Proceedings of the ACM Annual Conference*.
- [167] Gilbert Wondracek, Paolo Milani Comparetti, Christopher Kruegel, Engin Kirda, and Scuola Superiore S Anna. 2008. Automatic Network Protocol Analysis.. In *Proceedings of the Network and Distributed System Security Symposium*.
- [168] Wai Kin Wong, Daoyuan Wu, Huaijin Wang, Zongjie Li, Zhibo Liu, Shuai Wang, Qiye Tang, Sen Nie, and Shi Wu. 2025. Decllm: Llm-augmented recompilable decompilation for enabling programmatic use of decompiled code. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1841–1864.
- [169] Yuhao Wu, Jinwen Wang, Yujie Wang, Shixuan Zhai, Zihan Li, Yi He, Kun Sun, Qi Li, and Ning Zhang. 2024. Your Firmware Has Arrived: A Study of Firmware Update Vulnerabilities. In *Proceedings of the USENIX Security Symposium*.
- [170] Danning Xie, Zhuo Zhang, Nan Jiang, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. 2024. ReSym: Harnessing LLMs to Recover Variable and Data Structure Symbols from Stripped Binaries. In *Proceedings of the on ACM SIGSAC Conference on Computer and Communications Security*.
- [171] Jiaqi Xiong, Guoqiang Chen, Kejiang Chen, Han Gao, Shaoyin Cheng, and Weiming Zhang. 2023. Hext5: Unified pre-training for stripped binary code information inference. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*.
- [172] Xiangzhe Xu, Zhuo Zhang, Zian Su, Ziyang Huang, Shiwei Feng, Yapeng Ye, Nan Jiang, Danning Xie, Siyuan Cheng, Lin Tan, et al. 2025. Unleashing the power of generative model in recovering variable names from stripped binary. In *Proceedings of the Network and Distributed System Security Symposium*.
- [173] Khaled Yakdan. 2018. *A human-centric approach for binary code decompilation*. Ph. D. Dissertation. University of Bonn.
- [174] Khaled Yakdan, Sergej Dechand, Elmar Gerhards-Padilla, and Matthew Smith. 2016. Helping Johnny to Analyze Malware: A Usability-Optimized Decompiler and Malware Analysis User Study. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [175] Khaled Yakdan, Sebastian Eschweiler, and Elmar Gerhards-Padilla. 2013. Recompile: A Decompilation Framework for Static Analysis of Binaries. In *Proceedings of the International Conference on Malicious and Unwanted Software*.
- [176] Khaled Yakdan, Sebastian Eschweiler, Elmar Gerhards-Padilla, and Matthew Smith. 2015. No more gotos: decompilation using pattern-independent control-flow structuring and semantic-preserving transformations. In *Proceedings of the Network and Distributed System Security Symposium*.
- [177] Qiuchen Yan and Stephen McCamant. 2014. *Conservative signed/unsigned type inference for binaries using minimum cut*. Technical Report 14-006. Department of Computer Science and Engineering, University of Minnesota.
- [178] Zao Yang and Stefan Nagy. 2025. {Bin2Wrong}: a Unified Fuzzing Framework for Uncovering Semantic Errors in {Binary-to-C} Decompilers. In *Proceedings of the USENIX Annual Technical Conference*.
- [179] Z3Prover Development Team. 2025. Z3 theorem prover. <https://github.com/Z3Prover/z3>.
- [180] Junyuan Zeng, Yangchun Fu, Kenneth A Miller, Zhiqiang Lin, Xiangyu Zhang, and Dongyan Xu. 2013. Obfuscation resilient binary code reuse through trace-oriented programming. In *Proceedings of the ACM SIGSAC conference on Computer and communications security*.
- [181] Mingwei Zhang, Aravind Prakash, Xiaolei Li, Zhenkai Liang, and Heng Yin. 2012. Identifying and analyzing pointer misuses for sophisticated memory-corruption exploit diagnosis. (2012).
- [182] Runze Zhang, Ying Cao, Ruigang Liang, Peiwei Hu, and Kai Chen. 2024. Optimizing Decompiler Output by Eliminating Redundant Data Flow in Self-Recursive Inlining. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution*.
- [183] Zhuo Zhang, Yapeng Ye, Wei You, Guanhong Tao, Wen-chuan Lee, Yonghwi Kwon, Yousra Aafer, and Xiangyu Zhang. 2021. Osprey: Recovery of Variable and Data Structure via Probabilistic Analysis for Stripped Binary. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [184] Chang Zhu, Ziyang Li, Anton Xue, Ati Priya Bajaj, Wil Gibbs, Yibo Liu, Rajeev Alur, Tiffany Bao, Hanjun Dai, Adam Doupe, et al. 2024. TYGR: Type Inference on Stripped Binaries Using Graph Neural Networks. In *Proceedings of the USENIX Security Symposium*.
- [185] Muqi Zou, Arslan Khan, Ruoyu Wu, Han Gao, Antonio Bianchi, and Dave Jing Tian. 2024. D-Helix: A Generic Decompiler Testing Framework Using Symbolic Differentiation. In *Proceedings of USENIX Security Symposium*.
- [186] Lukáš Ďurfina, Jakub Kroutek, Petr Zemek, Dušan Kolář, Tomáš Hruška, Karel Mašarik, and Alexander Meduna. 2011. Design of a retargetable decompiler for a static platform-independent malware analysis. In *Proceedings of the International Conference on Information Security and Assurance*.